Phase 0 (Foundation) Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Stand up the multi-tenant foundation (Dockerized dev environment, Users & Company schema, PostgreSQL Row-Level Security, tenant context propagation, auth, invitations, and audit logging) that every later Builders Stream module depends on, with an automated tenant-isolation test suite enforced in CI.

Architecture: Decoupled FastAPI backend (Python 3.12, async SQLAlchemy 2.0 + Alembic) behind a `TenantMiddleware`, PostgreSQL with Row-Level Security as the authoritative isolation mechanism, and a minimal Next.js frontend that only proves end-to-end connectivity in this phase. Full rationale is in `docs/03-technical-architecture.md` and the schema in `docs/04-database-schema.md`.

Tech Stack: FastAPI, SQLAlchemy 2.0 (async, asyncpg driver), Alembic, PostgreSQL 16, PyJWT, argon2-cffi, pytest + pytest-asyncio + httpx, Docker Compose, GitHub Actions, Next.js 16 (TypeScript).

Critical design decisions locked in by this plan

These are not implementation details to improvise later — get them wrong and tenant isolation silently doesn't work. Read this section before touching any task below.

1. **The app must connect as a non-owner database role.** PostgreSQL Row-Level Security policies are **not enforced against the table owner or a superuser** by default. If the FastAPI app connects using the same role that ran the migrations (commonly `postgres`), every RLS policy in the system is silently bypassed and nothing is actually isolated. Task 5 creates a dedicated `app_user` role with `SELECT/INSERT/UPDATE/DELETE` grants (no ownership), and the app's runtime `DATABASE_URL` connects as `app_user` while Alembic migrations run as the superuser/owner via a separate `MIGRATIONS_DATABASE_URL`.

2. **New-company INSERT is bootstrap-safe by construction, not by bypassing RLS.** Company IDs are generated in Python (`uuid.uuid4()`), not by a database default, so the app knows the new company's ID *before* inserting it and can `SET LOCAL app.current_tenant` to that ID inside the same transaction, in order:

insert `companies` row (allowed because its `parent_id IS NULL`) → set tenant context to the new ID → insert `company_users`/`audit_log` rows (allowed because the row that makes them valid now exists in the same transaction). See Task 5's INSERT policies and Task 9.

3. **`X-Tenant-ID` is a claim, not a grant — membership is verified before it's trusted.** Trusting a client-supplied tenant header outright would let any authenticated user read another company's data just by sending that company's ID in a header. `get_current_user` (Task 11) always verifies a `company_users` row exists for `(user_id, claimed_tenant)` before setting `app.current_tenant`. That membership lookup itself has a bootstrap problem (it queries an RLS-protected table before tenant context is established) — solved with a second, independently-scoped RLS policy keyed on `app.current_user_id` rather than `app.current_tenant` (Task 5, "self-membership" policy). Two Postgres permissive policies on the same command are OR'd together, so this doesn't weaken isolation — a user can only ever see their **own** membership rows through this path.

4. **Refresh-token rotation is explicitly out of scope for this plan.** `docs/07-security-compliance.md` specifies short-lived access tokens with refresh rotation as the target end state. This plan issues a single 60-minute access token with no refresh flow, so the login session simply expires and the user re-authenticates. This is a known, deliberate simplification — track it as follow-up work before real subscriber data is on the platform, not a Phase-0 blocker.

5. **`get_all_descendant_ids()` must be `SECURITY DEFINER`, or RLS recurses infinitely for the real runtime role.** The function queries `companies`, and `companies`' own RLS policies call the function — for any caller that isn't the table owner (i.e. `app_user`, the actual role the app connects as), that's unbounded recursion: the function's internal query re-triggers the policy that invoked it, forever, until Postgres raises `stack depth limit exceeded`. This is invisible if you only test as the `postgres` superuser (superusers bypass RLS, so the recursive policy call never fires) — which is exactly why Task 5's own verification steps test the `app_user` role directly, not just check that RLS is "enabled." `SECURITY DEFINER` makes the function's internal traversal run as its owner (`postgres`), bypassing RLS only for that internal scan; the outer policies still gate everything real callers can see. Because a `SECURITY DEFINER` function bypasses RLS for anyone who can call it, `EXECUTE` is revoked from `PUBLIC` and granted only to `app_user`, matching the narrow-by-default posture of the rest of this schema. Caught empirically during Task 5 implementation, not anticipated in the original design — see Task 5's migration for the full mechanism.

6. **Every `UPDATE`-permitting RLS policy needs its own `WITH CHECK`, not just `USING`.** `USING` alone only gates which *existing* rows a caller may target — it says nothing about the *new* values being written. `companies.tenant_update` originally had only `USING`, which meant a tenant could `UPDATE companies SET parent_id = '<other-tenant>' WHERE id = '<own-company>'` and successfully re-parent its own company into a completely unrelated tenant's tree — a full tenant-boundary bypass via UPDATE, even though INSERT and SELECT on the same table were correctly locked down. Empirically confirmed exploitable (a bare cross-tenant re-parent `UPDATE` succeeded, and the row correctly vanished from the original tenant's view immediately after) before a `WITH CHECK` mirroring `tenant_insert`'s was added. Not reachable through any endpoint this plan currently defines (Task 13 only adds company-creation, not re-parenting), but it's the isolation backstop every later task is told to trust — caught during Task 5's code-quality review, not anticipated in the original design.

7. **Every `current_setting('app.x', true)::uuid` cast in an RLS policy must guard against `''`, not just `NULL`.** A custom ("placeholder") GUC, once set even once via `SET LOCAL`/`set_config(..., is_local=true)` on a given physical connection, does not revert to NULL when that transaction ends — `current_setting(name, true)` instead returns `''` for the rest of that connection's lifetime, even across later, unrelated transactions that never touch the setting themselves. Casting `''::uuid` raises `invalid input syntax for type uuid`, an unhandled error (a 500), not a policy evaluating to false. Because a connection pool reuses physical connections across unrelated logical requests, this bites in exactly the way that's hardest to catch in isolated testing: a request that never sets `app.current_tenant` (e.g. `login()`, which only sets `app.current_user_id`) can be served a connection previously "poisoned" by an earlier request that did set it and commit (e.g. `register()`). This went undetected through all of Task 5's and Task 9's own testing because every manual verification used a *fresh* connection per test — it only surfaced when Task 10 chained register→login through the app's real connection-pooled `session_scope()` for the first time. Fixed by wrapping every such cast as `NULLIF(current_setting('app.x', true), '')::uuid`, so a poisoned `''` becomes a real `NULL` before the cast, and the policy correctly denies access instead of erroring. Verified empirically against live Postgres in both the broken and fixed forms.

8. **`get_current_user` must defer its commit past `yield`, or every route handler downstream loses tenant context.** `set_current_user`/`set_current_tenant` are transaction-scoped (design decision #7). The original draft called `session.commit()` before

returning `CurrentUser`, ending the transaction — meaning by the time a route handler (Task 12+) reused `CurrentUser.session` for its own query, `app.current_tenant` had already reverted to `''`/NULL and RLS would deny access to the caller's *own* data. This is a FastAPI "dependency with yield": the fix restructures `get_current_user` as an async generator that `yields` `CurrentUser` mid-transaction and only commits (or rolls back, on exception) *after* the route handler returns, so the transaction — and the tenant context set within it — stays alive for the whole request. Verified empirically both ways: the eager-commit version returns zero rows for a route handler querying the caller's own company; the deferred-commit version returns it correctly, with no connection leaks or deadlocks across repeated success/failure requests. An earlier attempt at "just remove the commit" without the `yield`-based teardown left the transaction open indefinitely, which deadlocked the test suite's own cleanup fixture — the `yield` structure (not merely deferring the commit) is what makes this safe.

9. **`invitations` needed its own bootstrap RLS policy — `accept_invitation`'s pre-tenant-context probe, as originally drafted in this plan, 404s on every invitation, valid or not.** `accept_invitation` (Task 14) runs before the invitee has any tenant membership, so — exactly like the `company_users` membership lookup in design decision #3 — its first query (`SELECT company_id FROM invitations WHERE id = :id`, used to discover which tenant to `SET` before re-querying the full row) has a chicken-and-egg problem: it must run before `app.current_tenant` can be set to anything. Unlike `company_users`, `invitations` (migration 0001) was only ever given the single `tenant_isolation` FOR ALL policy, with no bootstrap-permissive companion policy. With no tenant context set, `tenant_isolation`'s `USING` clause reduces to `company_id IN (SELECT ... FROM get_all_descendant_ids(NULL))`, which is always empty — so the probe query returns nothing for every invitation, and the endpoint 404s unconditionally, regardless of whether the invitation is real, expired, or already accepted. This is not a hypothetical: it was reproduced empirically by implementing Task 14 exactly as originally specified and running the test suite against the live Postgres container — `test_accept_invitation_creates_user_and_membership` and `test_accept_expired_invitation_is_rejected` both failed with `404` instead of `200`/`410`. Fixed with the same established pattern as design decision #3: a second, narrowly-scoped permissive RLS policy, `invitation_probe` (migration `0002_invitation_probe_policy.py`), `FOR SELECT ... USING (id = NULLIF(current_setting('app.probing_invitation_id', true), '')::uuid)`. Where `self_membership` keys on `app.current_user_id` (a fact the caller already proved via a valid JWT), `invitation_probe` keys

on a new GUC, `app.probing_invitation_id`, which `accept_invitation` now sets (via the new `set_invitation_probe()` helper in `app/db.py`) to the exact `invitation_id` from the URL path, immediately before the probe query. Because Postgres ORs permissive policies of the same command together, this doesn't weaken isolation on any other query path — it only ever allows a session to see the single invitation row whose id it already explicitly supplied; a wrong/unknown id still correctly finds nothing. Verified empirically two ways: (1) the full test suite (38 tests) passes with the fix, including both previously-failing tests; (2) a direct `asyncpg` probe against `app_user` with the GUC set to one invitation's id confirmed it can see *only* that row — not a second, otherwise-identical invitation belonging to the same company, and not via a bare `SELECT * FROM invitations` with no `WHERE` clause either.

10. **`docker-compose.yml`'s `NEXT_PUBLIC_API_URL` for the frontend service must use the Compose network hostname (`http://backend:8000`), not `localhost`.** The original Task 1 scaffolding set it to `http://localhost:8000`. `app/page.tsx`'s health-check fetch (Task 18) runs server-side, inside the frontend container — from there, `localhost` resolves to the frontend container itself, not the backend, so the fetch would always fail and the page would always render "Backend status: unreachable," silently defeating the entire point of Task 18 (proving the Compose topology is wired correctly). Fixed to `http://backend:8000`, matching the same Docker-internal-DNS hostname convention already used for `DATABASE_URL`. Verified by bringing up the full stack and confirming the frontend actually renders "Backend status: ok". Separately: Next.js 16 / React 19's SSR inserts an HTML comment marker between static and dynamic text segments in a Server Component like this one, so the raw HTTP response body is literally `Backend status: <!-- -->ok`, not a contiguous string — Task 19's smoke-test assertion accounts for this by stripping HTML comments before checking, rather than asserting on the literal substring `"Backend status: ok"` directly.

Regression Testing Policy

Each task's own verification step only proves *that task's* new code works in isolation. Nothing in the per-task steps re-checks that a later task hasn't silently broken an earlier one — Task 4's model changes could break Task 2's health check, Task 13's nested-hierarchy endpoint could break Task 12's isolation guarantees, and so on. Two things close that gap:

1. **From the first task that has a real pytest suite onward (Task 9, which adds `conftest.py`), every subsequent task's verification step includes running the *entire* suite — `pytest -v` from `backend/`**

with no path filter — not just the new task's test file. A task is not done if the full suite has a regression, even one unrelated to that task's own code. If a task's changes break an earlier test, the earlier test's expectations don't automatically win — figure out which one is actually wrong and fix that one, but never leave both in a state where the suite is red.

2. **Task 19 (after Task 18, before Phase 0 is considered complete) runs a genuine end-to-end regression pass against the real, fully-assembled Docker Compose stack** — actual containers, actual network calls over real HTTP, not the in-process ASGI transport every other task's tests use. This is the only point in the plan that exercises the system the way a real client actually would, including the two hostname contexts (`postgres` from inside the Docker network vs. `localhost` from the host) actually working together correctly.

This is deliberately layered, not redundant: fast in-process tests give quick feedback per task, and the slower full-stack pass at the end catches the class of bug that only shows up when everything runs together for real (wiring, hostnames, container startup ordering, environment variable propagation).

File Structure

```
docker-compose.yml
.env.example
backend/
  Dockerfile
  pyproject.toml
  pytest.ini
  alembic.ini
  migrations/
    env.py
    script.py.mako
    versions/
      0001_initial_schema.py
      0002_invitation_probe_policy.py
  app/
    __init__.py
    main.py
    config.py
    db.py
    core/
      __init__.py
      context.py
      middleware.py
      security.py
      deps.py
```

```
    models/
      __init__.py
      base.py
      company.py
      user.py
      audit.py
    schemas/
      __init__.py
      auth.py
      company.py
      invitation.py
    routers/
      __init__.py
      auth.py
      companies.py
      invitations.py
    services/
      __init__.py
      audit.py
  tests/
    __init__.py
    conftest.py
    test_health.py
    test_auth.py
    test_tenant_isolation.py
    test_invitations.py
    test_rls_policy_regression.py
frontend/
  Dockerfile
  package.json
  tsconfig.json
  next.config.ts
  app/
    layout.tsx
    page.tsx
.github/
  workflows/
    backend-ci.yml
```

Task 1: Repo & Docker Compose Scaffolding

Files: - Create: `.env.example` - Create: `docker-compose.yml` - Create: `backend/Dockerfile` - Create: `frontend/Dockerfile`

- ☐ **Step 1: Create the environment template**

`.env.example`:

```
# Postgres superuser — used ONLY by Alembic migrations, never by the
running app
POSTGRES_USER=postgres
POSTGRES_PASSWORD=devpassword
POSTGRES_DB=builders_stream

# App runtime connection — non-owner role created by migration 0001
(see design decision #1)
APP_DB_USER=app_user
APP_DB_PASSWORD=app_password

DATABASE_URL=postgresql+asyncpg://app_user:app_password@postgres:5432/
builders_stream
MIGRATIONS_DATABASE_URL=postgresql+asyncpg://postgres:devpassword@loca
lhost:5432/builders_stream
TEST_DATABASE_URL=postgresql+asyncpg://postgres:devpassword@localhost:
5432/builders_stream_test

JWT_SECRET=dev-only-secret-change-me
JWT_EXPIRE_MINUTES=60

REDIS_URL=redis://redis:6379/0
```

Hostname note: `DATABASE_URL` uses the Docker-network hostname `postgres` because it's read by the `backend` container at real runtime (Task 18), where service names resolve via Docker's internal DNS. `MIGRATIONS_DATABASE_URL` and `TEST_DATABASE_URL` use `localhost` instead, because Alembic (Task 5) and pytest (Task 9) both run directly on the host machine in this plan, not inside a container — from the host, only `localhost:5432` (published by `docker-compose.yml`'s `"5432:5432"` port mapping) resolves, `postgres` does not.

- ☐ **Step 2: Create the Docker Compose stack**

`docker-compose.yml`:

```yaml
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 5s
      timeout: 5s
```

```
      retries: 10

  redis:
    image: redis:7
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 5s
      retries: 10

  backend:
    build: ./backend
    env_file: .env
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    ports:
      - "8000:8000"
    volumes:
      - ./backend:/app
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

  frontend:
    build: ./frontend
    depends_on:
      - backend
    ports:
      - "3000:3000"
    volumes:
      - ./frontend:/app
      - /app/node_modules
    environment:
      NEXT_PUBLIC_API_URL: http://localhost:8000

volumes:
  pgdata:
```

- **☐ Step 3: Create the backend Dockerfile**

backend/Dockerfile:

```
FROM python:3.12-slim

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends gcc
```

```
libpq-dev && rm -rf /var/lib/apt/lists/*

COPY pyproject.toml ./
RUN pip install --no-cache-dir -e .

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

- ☐ **Step 4: Create the frontend Dockerfile**

`frontend/Dockerfile`:

```
FROM node:22-slim

WORKDIR /app

COPY package.json ./
RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "run", "dev"]
```

- ☐ **Step 5: Verify Postgres and Redis start cleanly**

Run:

```
cp .env.example .env
docker compose up -d postgres redis
docker compose ps
```

Expected: both `postgres` and `redis` show `healthy` status within ~15 seconds.

- ☐ **Step 6: Commit**

```
git add .env.example docker-compose.yml backend/Dockerfile
frontend/Dockerfile
git commit -m "chore: add Docker Compose dev environment scaffolding"
```

Task 2: Backend Project Scaffolding (FastAPI skeleton)

Files: - Create: `backend/pyproject.toml` - Create: `backend/pytest.ini` - Create: `backend/app/__init__.py` - Create: `backend/app/main.py` - Create: `backend/app/config.py` - Test: `backend/tests/__init__.py`, `backend/tests/test_health.py`

- ☐ **Step 1: Write `pyproject.toml`**

```
[project]
name = "builders-stream-backend"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
    "fastapi>=0.115,<0.116",
    "uvicorn[standard]>=0.32,<0.33",
    "sqlalchemy[asyncio]>=2.0,<2.1",
    "asyncpg>=0.30,<0.31",
    "alembic>=1.14,<1.15",
    "pydantic>=2.9,<3.0",
    "pydantic-settings>=2.6,<3.0",
    "pyjwt>=2.9,<3.0",
    "argon2-cffi>=23.1,<24.0",
    "email-validator>=2.2,<3.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.3,<9.0",
    "pytest-asyncio>=0.24,<0.25",
    "httpx>=0.27,<0.28",
]

[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[tool.setuptools.packages.find]
include = ["app*"]
```

- ☐ **Step 2: Write `pytest.ini`**

```
[pytest]
asyncio_mode = auto
testpaths = tests
```

- ☐ **Step 3: Write `app/config.py`**

```
from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    database_url: str
    migrations_database_url: str
    test_database_url: str
    jwt_secret: str
    jwt_expire_minutes: int = 60
```

```
    redis_url: str = "redis://localhost:6379/0"


settings = Settings()
```

- ☐ **Step 4: Write `app/main.py`**

```
from fastapi import FastAPI

app = FastAPI(title="Builders Stream API", version="0.1.0")


@app.get("/health")
async def health() -> dict:
    return {"status": "ok"}
```

- ☐ **Step 5: Write the failing test**

`backend/tests/__init__.py` (empty file).

`backend/tests/test_health.py`:

```
from httpx import AsyncClient, ASGITransport

from app.main import app


async def test_health_returns_ok():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport,
base_url="http://test") as client:
        response = await client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "ok"}
```

- ☐ **Step 6: Install dependencies and run the test**

Run:

```
cd backend
pip install -e ".[dev]"
pytest tests/test_health.py -v
```

Expected: `test_health_returns_ok PASSED` (this endpoint already exists, so it should pass immediately — this step exists to confirm your local environment is wired correctly before moving on).

- ☐ **Step 7: Commit**

```
git add backend/pyproject.toml backend/pytest.ini
backend/app/__init__.py backend/app/main.py backend/app/config.py
backend/tests/
git commit -m "feat: scaffold FastAPI backend with health check"
```

Task 3: Settings Wiring for the Two-Tier DB Connection

Files: - Modify: `.env` (local only, not committed — created from `.env.example` in Task 1) - Create: `backend/app/db.py`

- ☐ **Step 1: Write `app/db.py`**

```
from contextlib import asynccontextmanager
from typing import AsyncIterator

from sqlalchemy import text
from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker,
create_async_engine

from app.config import settings

# Runtime engine: connects as the restricted `app_user` role (see
design decision #1).
# RLS policies are enforced against this connection.
engine = create_async_engine(settings.database_url,
pool_pre_ping=True)
SessionLocal = async_sessionmaker(engine, expire_on_commit=False,
class_=AsyncSession)


@asynccontextmanager
async def session_scope() -> AsyncIterator[AsyncSession]:
    async with SessionLocal() as session:
        yield session


async def set_current_user(session: AsyncSession, user_id: str) ->
None:
    """Scopes the self-membership RLS policy (design decision #3) to
this user for the
    remainder of the current transaction.

    Uses set_config(), not `SET LOCAL ... = :param`, because
PostgreSQL's SET/SET LOCAL
    grammar only accepts a literal in the value position — a bound
parameter there is a
    syntax error at the server, regardless of driver. set_config(name,
value, is_local)
    is a plain function call, so it accepts bound parameters normally;
is_local=true
    gives it the same transaction-scoped reset-on-commit/rollback
semantics as SET LOCAL.
    """
```

```
    await session.execute(
        text("SELECT set_config('app.current_user_id', :uid, true)"),
{"uid": user_id}
    )


async def set_current_tenant(session: AsyncSession, company_id: str) -
> None:
    """Scopes every tenant-isolation RLS policy to this company (and
its descendants)
    for the remainder of the current transaction. See
set_current_user's docstring for
    why this uses set_config() instead of SET LOCAL with a bound
parameter."""
    await session.execute(
        text("SELECT set_config('app.current_tenant', :cid, true)"),
{"cid": company_id}
    )
```

- ☐ **Step 2: Verify it imports cleanly**

Run:

```
cd backend
python -c "from app.db import engine, SessionLocal, session_scope,
set_current_user, set_current_tenant; print('ok')"
```

Expected: `ok` (no connection is attempted at import time — `create_async_engine` is lazy).

- ☐ **Step 3: Commit**

```
git add backend/app/db.py
git commit -m "feat: add async DB engine and tenant-context session
helpers"
```

Task 4: SQLAlchemy Models

Files: - Create: `backend/app/models/__init__.py` - Create: `backend/app/models/base.py` - Create: `backend/app/models/company.py` - Create: `backend/app/models/user.py` - Create: `backend/app/models/audit.py`

- ☐ **Step 1: Write the declarative base and shared mixin**

`backend/app/models/base.py`:

```
import uuid
from datetime import datetime, timezone

from sqlalchemy import DateTime
```

```
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column


class Base(DeclarativeBase):
    pass


def new_uuid() -> uuid.UUID:
    return uuid.uuid4()


def utcnow() -> datetime:
    return datetime.now(timezone.utc)


class UUIDPKMixin:
    id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
primary_key=True, default=new_uuid)


class TimestampMixin:
    created_at: Mapped[datetime] =
mapped_column(DateTime(timezone=True), default=utcnow)
```

- **☐ Step 2: Write the Company model**

backend/app/models/company.py:

```
import uuid

from sqlalchemy import Boolean, ForeignKey, String
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column

from app.models.base import Base, TimestampMixin, UUIDPKMixin


class Company(Base, UUIDPKMixin, TimestampMixin):
    __tablename__ = "companies"

    parent_id: Mapped[uuid.UUID | None] = mapped_column(
        UUID(as_uuid=True), ForeignKey("companies.id",
ondelete="CASCADE"), nullable=True
    )
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)
```

- **☐ Step 3: Write the User, CompanyUser, and Invitation models**

backend/app/models/user.py:

```
import uuid
from datetime import datetime

from sqlalchemy import CheckConstraint, DateTime, ForeignKey, String
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column

from app.models.base import Base, TimestampMixin, UUIDPKMixin

VALID_ROLES = ("admin", "project_manager", "field_crew", "accountant",
"client")

# Rendered to match migrations/versions/0001_initial_schema.py's
literal SQL exactly
# (no spaces after commas) so `alembic revision --autogenerate` never
reports a
# spurious constraint diff between the ORM model and the real
database.
_ROLE_CHECK_SQL = "role IN (" + ",".join(f"'{role}'" for role in
VALID_ROLES) + ")"


class User(Base, UUIDPKMixin, TimestampMixin):
    __tablename__ = "users"

    email: Mapped[str] = mapped_column(String(255), unique=True,
nullable=False)
    password_hash: Mapped[str] = mapped_column(String, nullable=False)
    full_name: Mapped[str | None] = mapped_column(String(255),
nullable=True)


class CompanyUser(Base):
    __tablename__ = "company_users"

    company_id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True), ForeignKey("companies.id",
ondelete="CASCADE"), primary_key=True
    )
    user_id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True), ForeignKey("users.id",
ondelete="CASCADE"), primary_key=True
    )
    role: Mapped[str] = mapped_column(String(50), nullable=False)
    created_at: Mapped[datetime] =
mapped_column(DateTime(timezone=True))
```

```
    __table_args__ = (CheckConstraint(_ROLE_CHECK_SQL,
name="ck_company_users_role"),)


class Invitation(Base, UUIDPKMixin):
    __tablename__ = "invitations"

    company_id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True), ForeignKey("companies.id",
ondelete="CASCADE"), nullable=False
    )
    email: Mapped[str] = mapped_column(String(255), nullable=False)
    role: Mapped[str] = mapped_column(String(50), nullable=False)
    expires_at: Mapped[datetime] =
mapped_column(DateTime(timezone=True), nullable=False)
    accepted_at: Mapped[datetime | None] =
mapped_column(DateTime(timezone=True), nullable=True)

    __table_args__ = (CheckConstraint(_ROLE_CHECK_SQL,
name="ck_invitations_role"),)
```

- **☐ Step 4: Write the AuditLog model**

backend/app/models/audit.py:

```
import uuid

from sqlalchemy import ForeignKey, String
from sqlalchemy.dialects.postgresql import JSONB, UUID
from sqlalchemy.orm import Mapped, mapped_column

from app.models.base import Base, TimestampMixin, UUIDPKMixin


class AuditLog(Base, UUIDPKMixin, TimestampMixin):
    __tablename__ = "audit_log"

    # No ondelete here (defaults to RESTRICT), unlike every other
company_id FK in
    # this file. docs/07-security-compliance.md requires a minimum 7-
year audit log
    # retention and states no code path ever deletes audit entries —
CASCADE would
    # let deleting a company silently destroy its own audit trail,
which directly
    # contradicts that policy. RESTRICT means a company can't be
deleted while it
    # still has audit history, which is the correct failure mode until
Phase 0 (which
    # has no company-delete endpoint at all) grows one with an
```

```
explicit retention story.
    company_id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True), ForeignKey("companies.id"), nullable=False
    )
    actor_id: Mapped[uuid.UUID | None] = mapped_column(
        UUID(as_uuid=True), ForeignKey("users.id"), nullable=True
    )
    action: Mapped[str] = mapped_column(String(100), nullable=False)
    entity_type: Mapped[str] = mapped_column(String(50),
nullable=False)
    entity_id: Mapped[uuid.UUID] = mapped_column(UUID(as_uuid=True),
nullable=False)
    log_metadata: Mapped[dict | None] = mapped_column(JSONB,
nullable=True)
```

- **☐ Step 5: Write `app/models/__init__.py` so Alembic can discover all models**

```
from app.models.base import Base
from app.models.company import Company
from app.models.user import User, CompanyUser, Invitation
from app.models.audit import AuditLog

__all__ = ["Base", "Company", "User", "CompanyUser", "Invitation",
"AuditLog"]
```

- **☐ Step 6: Verify the models import cleanly**

Run:

```
cd backend
python -c "from app.models import Base, Company, User, CompanyUser,
Invitation, AuditLog; print(sorted(Base.metadata.tables.keys()))"
```

Expected: `['audit_log', 'companies', 'company_users', 'invitations', 'users']`

- **☐ Step 7: Commit**

```
git add backend/app/models/
git commit -m "feat: add SQLAlchemy models for companies, users, and
audit log"
```

Task 5: Alembic Migration — Schema, App Role, and Row-Level Security

This is the most important task in the plan. Re-read “Critical design decisions” above before writing this migration.

Files: - Create: `backend/alembic.ini` - Create: `backend/migrations/env.py` - Create: `backend/migrations/script.py.mako` - Create: `backend/migrations/versions/0001_initial_schema.py`

- ☐ **Step 1: Write `alembic.ini`**

```
[alembic]
script_location = migrations
prepend_sys_path = .

[loggers]
keys = root,sqlalchemy,alembic

[logger_root]
level = WARNING
handlers = console

[logger_sqlalchemy]
level = WARNING
handlers =
qualname = sqlalchemy.engine

[logger_alembic]
level = INFO
handlers =
qualname = alembic

[handlers]
keys = console

[handler_console]
class = StreamHandler
args = (sys.stderr,)
formatter = generic

[formatters]
keys = generic

[formatter_generic]
format = %(levelname)-5.5s [%(name)s] %(message)s
```

- ☐ **Step 2: Write `migrations/env.py`**

```
import asyncio
from logging.config import fileConfig

from alembic import context
from sqlalchemy.ext.asyncio import create_async_engine

from app.config import settings
from app.models import Base
```

```
config = context.config
if config.config_file_name is not None:
    fileConfig(config.config_file_name)

target_metadata = Base.metadata


def run_migrations_offline() -> None:
    context.configure(
        url=settings.migrations_database_url,
        target_metadata=target_metadata,
        literal_binds=True,
    )
    with context.begin_transaction():
        context.run_migrations()


def do_run_migrations(connection) -> None:
    context.configure(connection=connection,
target_metadata=target_metadata)
    with context.begin_transaction():
        context.run_migrations()


async def run_migrations_online() -> None:
    connectable =
create_async_engine(settings.migrations_database_url)
    async with connectable.connect() as connection:
        await connection.run_sync(do_run_migrations)
    await connectable.dispose()


if context.is_offline_mode():
    run_migrations_offline()
else:
    asyncio.run(run_migrations_online())
```

- ☐ **Step 3: Write `migrations/script.py.mako`**

```
"""${message}

Revision ID: ${up_revision}
Revises: ${down_revision | comma,n}
Create Date: ${create_date}

"""
from alembic import op
import sqlalchemy as sa
${imports if imports else ""}
```

```
revision = ${repr(up_revision)}
down_revision = ${repr(down_revision)}
branch_labels = ${repr(branch_labels)}
depends_on = ${repr(depends_on)}


def upgrade() -> None:
    ${upgrades if upgrades else "pass"}


def downgrade() -> None:
    ${downgrades if downgrades else "pass"}
```

- **☐ Step 4: Write the initial migration**

backend/migrations/versions/0001_initial_schema.py:

```
"""Initial schema: companies, users, company_users, invitations,
audit_log,
app_user role, and Row-Level Security policies.

Revision ID: 0001
Revises:
Create Date: 2026-07-07

"""
from alembic import op
import sqlalchemy as sa
from sqlalchemy.dialects.postgresql import JSONB, UUID

revision = "0001"
down_revision = None
branch_labels = None
depends_on = None


def upgrade() -> None:
    op.execute("CREATE EXTENSION IF NOT EXISTS pgcrypto")

    # --- Tables
------------------------------------------------------------
    op.create_table(
        "companies",
        sa.Column("id", UUID(as_uuid=True), primary_key=True),
        sa.Column("parent_id", UUID(as_uuid=True),
sa.ForeignKey("companies.id", ondelete="CASCADE"), nullable=True),
        sa.Column("name", sa.String(255), nullable=False),
        sa.Column("is_active", sa.Boolean, nullable=False,
server_default=sa.true()),
```

```
        sa.Column("created_at", sa.DateTime(timezone=True),
nullable=False, server_default=sa.func.now()),
    )
    op.create_index("idx_companies_parent_id", "companies",
["parent_id"])

    op.create_table(
        "users",
        sa.Column("id", UUID(as_uuid=True), primary_key=True),
        sa.Column("email", sa.String(255), nullable=False,
unique=True),
        sa.Column("password_hash", sa.String, nullable=False),
        sa.Column("full_name", sa.String(255), nullable=True),
        sa.Column("created_at", sa.DateTime(timezone=True),
nullable=False, server_default=sa.func.now()),
    )

    op.create_table(
        "company_users",
        sa.Column("company_id", UUID(as_uuid=True),
sa.ForeignKey("companies.id", ondelete="CASCADE"), primary_key=True),
        sa.Column("user_id", UUID(as_uuid=True),
sa.ForeignKey("users.id", ondelete="CASCADE"), primary_key=True),
        sa.Column("role", sa.String(50), nullable=False),
        sa.Column("created_at", sa.DateTime(timezone=True),
nullable=False, server_default=sa.func.now()),
        sa.CheckConstraint(
            "role IN
('admin','project_manager','field_crew','accountant','client')",
            name="ck_company_users_role",
        ),
    )

    op.create_table(
        "invitations",
        sa.Column("id", UUID(as_uuid=True), primary_key=True),
        sa.Column("company_id", UUID(as_uuid=True),
sa.ForeignKey("companies.id", ondelete="CASCADE"), nullable=False),
        sa.Column("email", sa.String(255), nullable=False),
        sa.Column("role", sa.String(50), nullable=False),
        sa.Column("expires_at", sa.DateTime(timezone=True),
nullable=False),
        sa.Column("accepted_at", sa.DateTime(timezone=True),
nullable=True),
        sa.CheckConstraint(
            "role IN
('admin','project_manager','field_crew','accountant','client')",
            name="ck_invitations_role",
        ),
    )
```

```
    op.create_table(
        "audit_log",
        sa.Column("id", UUID(as_uuid=True), primary_key=True),
        # No ondelete (defaults to RESTRICT) — see the matching
comment on the
        # AuditLog ORM model (Task 4): CASCADE here would violate the
audit log's
        # documented 7-year, never-deleted retention policy.
        sa.Column("company_id", UUID(as_uuid=True),
sa.ForeignKey("companies.id"), nullable=False),
        sa.Column("actor_id", UUID(as_uuid=True),
sa.ForeignKey("users.id"), nullable=True),
        sa.Column("action", sa.String(100), nullable=False),
        sa.Column("entity_type", sa.String(50), nullable=False),
        sa.Column("entity_id", UUID(as_uuid=True), nullable=False),
        sa.Column("log_metadata", JSONB, nullable=True),
        sa.Column("created_at", sa.DateTime(timezone=True),
nullable=False, server_default=sa.func.now()),
    )
    op.create_index("idx_audit_log_company_created", "audit_log",
["company_id", "created_at"])

    # --- Recursive descendant lookup (design decision #2/#3 depend on
this) -
    #
    # SECURITY DEFINER + a pinned search_path are required here, not
optional
    # hardening: this function queries `companies` internally, and
`companies`
    # has RLS enabled with a SELECT policy that itself calls this
function.
    # For any caller who is not the table owner (i.e. the real runtime
    # `app_user` role — see design decision #1), a plain (non-SECURITY
    # DEFINER) version of this function recurses infinitely: the
function's
    # internal SELECT on `companies` re-triggers the `tenant_select`
policy,
    # which calls this function again, forever, until Postgres raises
    # "stack depth limit exceeded". This was caught by directly
testing as
    # app_user during Task 5 implementation — every single query
against any
    # RLS-protected table failed until this fix was applied. It would
NOT
    # have been caught by running Step 5/6's verification as the
`postgres`
    # superuser alone, since superusers bypass RLS and never trigger
the
    # recursive policy call in the first place.
```

```
    #
    # Marking the function SECURITY DEFINER makes its body execute
with the
    # privileges of its owner (postgres, who also owns `companies` and
was
    # never granted FORCE ROW LEVEL SECURITY), so the internal
traversal
    # bypasses RLS entirely and terminates normally. This does NOT
weaken
    # tenant isolation: the outer RLS policies still gate what
`app_user`
    # can ultimately see/write via `... IN (SELECT id FROM
    # get_all_descendant_ids(...))` — only this function's own
internal scan
    # is exempted, and EXECUTE on it is restricted to app_user below
(not
    # left at Postgres's PUBLIC default, since a SECURITY DEFINER
function
    # bypasses RLS for anyone who can call it directly).
    op.execute(
        """
        CREATE OR REPLACE FUNCTION get_all_descendant_ids(root_id
UUID)
        RETURNS TABLE (id UUID)
        LANGUAGE sql
        STABLE
        SECURITY DEFINER
        SET search_path = public, pg_temp
        AS $$
            WITH RECURSIVE company_tree AS (
                SELECT c.id FROM companies c WHERE c.id = root_id
                UNION ALL
                SELECT c.id FROM companies c INNER JOIN company_tree
ct ON c.parent_id = ct.id
            )
            SELECT id FROM company_tree;
        $$;
        """
    )

    # --- Restricted application role (design decision #1)
-------------------
    op.execute(
        """
        DO $$
        BEGIN
            IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname =
'app_user') THEN
                CREATE ROLE app_user WITH LOGIN PASSWORD
'app_password';
```

```
            END IF;
        END
        $$;
        """
    )
    op.execute("GRANT USAGE ON SCHEMA public TO app_user")
    op.execute("GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN
SCHEMA public TO app_user")
    op.execute(
        "ALTER DEFAULT PRIVILEGES IN SCHEMA public "
        "GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO app_user"
    )
    # Postgres grants EXECUTE on new functions to PUBLIC by default.
Since
    # get_all_descendant_ids is SECURITY DEFINER (see the comment
above its
    # definition), ANY SQL running as app_user — not just the RLS
policy
    # engine's internal use of it — can call it directly with an
arbitrary
    # root_id and get back that root's descendant company ids,
regardless of
    # the caller's own tenant context. This is an accepted, narrow
residual:
    # it only leaks UUID parent/child relationships (no other
columns), and
    # app_user is the backend's single trusted connection role, not
something
    # end users get raw access to. Revoking PUBLIC and granting only
to
    # app_user keeps the surface as narrow as it can be while the
function
    # still does its job — it's not a full fix, since app_user itself
must
    # retain EXECUTE for the policies above to work.
    op.execute("REVOKE EXECUTE ON FUNCTION
get_all_descendant_ids(UUID) FROM PUBLIC")
    op.execute("GRANT EXECUTE ON FUNCTION get_all_descendant_ids(UUID)
TO app_user")

    # --- Row-Level Security
-------------------------------------------
    #
    # Every current_setting('app.x', true) cast below is wrapped in
    # NULLIF(..., '') before ::uuid — this is not defensive styling,
it fixes
    # a real bug found empirically via connection pooling. A custom
    # ("placeholder") GUC like app.current_tenant, once set even once
via
    # SET LOCAL / set_config(..., is_local=true) on a given physical
```

```
    # connection, does NOT revert to NULL when that transaction ends —
    # current_setting(name, true) instead returns '' (empty string)
for the
    # rest of that connection's life, even in later, unrelated
transactions
    # that never set it themselves. Casting '' directly to ::uuid
raises
    # invalid input syntax for type uuid: "", which is NOT the same as
the
    # policy evaluating to false — it's an unhandled error that
surfaces as a
    # 500. Because connection pools reuse physical connections across
    # unrelated logical requests, this bites intentionally: any
request that
    # never sets app.current_tenant (e.g. login(), which only sets
    # app.current_user_id) can be served a connection previously
"poisoned"
    # by an earlier request that did set it (e.g. register()) and
commit.
    # NULLIF(x, '') turns the poisoned '' back into a real NULL before
the
    # cast, so the policy correctly evaluates to false (access denied)
    # instead of raising. Verified against live Postgres: the un-
guarded cast
    # reproduces the error deterministically once a connection has
ever seen
    # a SET on that GUC; the guarded version returns NULL and
    # get_all_descendant_ids(NULL) correctly returns zero rows.
    for table in ("companies", "company_users", "invitations",
"audit_log"):
        op.execute(f"ALTER TABLE {table} ENABLE ROW LEVEL SECURITY")

    # companies: split SELECT/UPDATE from INSERT so a brand-new top-
level
    # company (parent_id IS NULL) can be created before any tenant
context
    # exists (design decision #2).
    op.execute(
        """
        CREATE POLICY tenant_select ON companies FOR SELECT
        USING (id IN (SELECT id FROM
get_all_descendant_ids(NULLIF(current_setting('app.current_tenant',
true), '')::uuid)))
        """
    )
    # WITH CHECK here is not optional. Without it, Postgres would fall
back to
    # reusing USING for the check — but USING only constrains which
existing row
    # a caller may target, never the NEW values being written to it.
```

```
That leaves
    # parent_id completely unvalidated on UPDATE: a session scoped to
tenant A
    # could UPDATE one of A's own companies to set parent_id to an
unrelated
    # tenant B's id, re-parenting it out of A's tree and into B's — a
full
    # tenant-boundary bypass via UPDATE, even though INSERT and SELECT
on this
    # same table are correctly locked down. Empirically confirmed
exploitable
    # (a bare `UPDATE companies SET parent_id = '<other-tenant>' WHERE
id =
    # '<own-company>'` succeeds) before this WITH CHECK was added. The
    # condition mirrors tenant_insert's: a same-tenant update
(parent_id
    # unchanged, or still within the caller's own tree) is unaffected,
since
    # the row's pre-update parent_id must already satisfy this same
predicate
    # for the row to have been visible/targetable via USING in the
first place.
    op.execute(
        """
        CREATE POLICY tenant_update ON companies FOR UPDATE
        USING (id IN (SELECT id FROM
get_all_descendant_ids(NULLIF(current_setting('app.current_tenant',
true), '')::uuid)))
        WITH CHECK (
            parent_id IS NULL
            OR parent_id IN (SELECT id FROM
get_all_descendant_ids(NULLIF(current_setting('app.current_tenant',
true), '')::uuid))
        )
        """
    )
    op.execute(
        """
        CREATE POLICY tenant_insert ON companies FOR INSERT
        WITH CHECK (
            parent_id IS NULL
            OR parent_id IN (SELECT id FROM
get_all_descendant_ids(NULLIF(current_setting('app.current_tenant',
true), '')::uuid))
        )
        """
    )

    # company_users: the ordinary tenant policy, PLUS a self-
membership
```

```
    # policy so a user can always discover their own memberships even
    # before app.current_tenant is set (design decision #3). Postgres
ORs
    # permissive policies of the same command together.
    op.execute(
        """
        CREATE POLICY tenant_isolation ON company_users FOR ALL
        USING (company_id IN (SELECT id FROM
get_all_descendant_ids(NULLIF(current_setting('app.current_tenant',
true), '')::uuid)))
        WITH CHECK (company_id IN (SELECT id FROM
get_all_descendant_ids(NULLIF(current_setting('app.current_tenant',
true), '')::uuid)))
        """
    )
    op.execute(
        """
        CREATE POLICY self_membership ON company_users FOR SELECT
        USING (user_id = NULLIF(current_setting('app.current_user_id',
true), '')::uuid)
        """
    )

    for table in ("invitations", "audit_log"):
        op.execute(
            f"""
            CREATE POLICY tenant_isolation ON {table} FOR ALL
            USING (company_id IN (SELECT id FROM
get_all_descendant_ids(NULLIF(current_setting('app.current_tenant',
true), '')::uuid)))
            WITH CHECK (company_id IN (SELECT id FROM
get_all_descendant_ids(NULLIF(current_setting('app.current_tenant',
true), '')::uuid)))
            """
        )


def downgrade() -> None:
    for table in ("invitations", "audit_log"):
        op.execute(f"DROP POLICY IF EXISTS tenant_isolation ON
{table}")
    op.execute("DROP POLICY IF EXISTS self_membership ON
company_users")
    op.execute("DROP POLICY IF EXISTS tenant_isolation ON
company_users")
    op.execute("DROP POLICY IF EXISTS tenant_insert ON companies")
    op.execute("DROP POLICY IF EXISTS tenant_update ON companies")
    op.execute("DROP POLICY IF EXISTS tenant_select ON companies")
    op.execute("DROP FUNCTION IF EXISTS get_all_descendant_ids(UUID)")
    op.drop_table("audit_log")
```

```
    op.drop_table("invitations")
    op.drop_table("company_users")
    op.drop_table("users")
    op.drop_table("companies")
```

- ☐ **Step 5: Run the migration against the dev database**

Run:

```
cd backend
cp ../.env.example ../.env   # if not already done in Task 1
docker compose up -d postgres
alembic upgrade head
```

Expected: no errors; last line resembles `Running upgrade  -> 0001, Initial schema...`.

- ☐ **Step 6: Verify RLS is actually enabled (not just tables created)**

Run:

```
docker compose exec postgres psql -U postgres -d builders_stream -c "\
d+ companies" | grep -i "row security"
```

Expected: output includes `Row security is enabled.` or similar (exact wording varies by psql version, but a row about row security must be present — if it's missing, `ENABLE ROW LEVEL SECURITY` did not take effect and Task 5 must be re-checked before continuing).

This check alone is not sufficient — it (and the `psql -U postgres` connection used to run it) only proves RLS is turned on, never that it actually works, because the `postgres` superuser bypasses RLS entirely and would never hit a policy-recursion bug even if one existed. The real runtime role is `app_user`. Verify against that role specifically:

```
docker compose exec postgres psql -U app_user -d builders_stream -c "
BEGIN;
SELECT set_config('app.current_tenant', gen_random_uuid()::text,
true);
SELECT count(*) FROM companies;
ROLLBACK;
"
```

Expected: `count = 0`, no error. If this instead raises `stack depth limit exceeded`, `get_all_descendant_ids()` is missing `SECURITY DEFINER` (see the comment above its definition in Step 4) — a plain version of that function recurses infinitely the moment a non-owner role queries `companies`, since the function's own internal query re-triggers the same RLS policy that called it. This is exactly the failure mode Step 6's superuser-only check cannot detect.

- ☐ **Step 7: Commit**

```
git add backend/alembic.ini backend/migrations/
git commit -m "feat: add initial schema migration with RLS-enforced
multi-tenancy"
```

Task 6: Tenant & User Context Propagation

Files: - Create: `backend/app/core/__init__.py` - Create: `backend/app/core/context.py` - Create: `backend/app/core/middleware.py` - Modify: `backend/app/main.py` - Test: `backend/tests/test_middleware.py`

- ☐ **Step 1: Write the context variables**

`backend/app/core/context.py`:

```
from contextvars import ContextVar
from typing import Optional

# Populated by TenantMiddleware from the request's JWT / X-Tenant-ID
header.
# This is a *claim*, not a verified grant — see design decision #3.
claimed_tenant_id_ctx: ContextVar[Optional[str]] =
ContextVar("claimed_tenant_id", default=None)
bearer_token_ctx: ContextVar[Optional[str]] =
ContextVar("bearer_token", default=None)
```

- ☐ **Step 2: Write the middleware**

`backend/app/core/middleware.py`:

```
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request

from app.core.context import bearer_token_ctx, claimed_tenant_id_ctx


class TenantMiddleware(BaseHTTPMiddleware):
    """Extracts the bearer token and the claimed tenant ID from the
raw request
    and makes them available via contextvars for the duration of the
request.
    Does NOT verify the claim — that happens in `get_current_user`
    (design decision #3), which has database access and this
middleware does not.
    """

    async def dispatch(self, request: Request, call_next):
        auth_header = request.headers.get("Authorization", "")
        token = auth_header.removeprefix("Bearer ").strip() if
```

```
auth_header.startswith("Bearer ") else None
        tenant_header = request.headers.get("X-Tenant-ID")

        token_reset = bearer_token_ctx.set(token)
        tenant_reset = claimed_tenant_id_ctx.set(tenant_header)
        try:
            return await call_next(request)
        finally:
            bearer_token_ctx.reset(token_reset)
            claimed_tenant_id_ctx.reset(tenant_reset)
```

- **☐ Step 3: Wire the middleware into the app**

Modify backend/app/main.py:

```
from fastapi import FastAPI

from app.core.middleware import TenantMiddleware

app = FastAPI(title="Builders Stream API", version="0.1.0")
app.add_middleware(TenantMiddleware)


@app.get("/health")
async def health() -> dict:
    return {"status": "ok"}
```

- **☐ Step 4: Write the failing test**

backend/tests/test_middleware.py:

```
from httpx import AsyncClient, ASGITransport

from app.core.context import bearer_token_ctx, claimed_tenant_id_ctx
from app.main import app


async def test_middleware_populates_context_from_headers():
    captured = {}

    @app.get("/_debug_context")
    async def debug_context():
        captured["token"] = bearer_token_ctx.get()
        captured["tenant"] = claimed_tenant_id_ctx.get()
        return {}

    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport,
base_url="http://test") as client:
        await client.get(
```

```
            "/_debug_context",
            headers={"Authorization": "Bearer abc123", "X-Tenant-ID":
"11111111-1111-1111-1111-111111111111"},
        )

    assert captured["token"] == "abc123"
    assert captured["tenant"] == "11111111-1111-1111-1111-
111111111111"


async def test_middleware_leaves_context_none_when_headers_absent():
    """Covers the branches the first test doesn't: no Authorization
header at
    all, and no X-Tenant-ID header. Both should resolve to None, not
an
    exception or a stale value — later code (Task 11's
get_current_user)
    treats None as "no claim" and falls back appropriately; a
regression here
    (e.g. someone breaks the `else None` ternary in middleware.py)
would
    silently propagate a wrong value instead of failing loudly."""
    captured = {}

    @app.get("/_debug_context_absent")
    async def debug_context_absent():
        captured["token"] = bearer_token_ctx.get()
        captured["tenant"] = claimed_tenant_id_ctx.get()
        return {}

    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport,
base_url="http://test") as client:
        await client.get("/_debug_context_absent")

    assert captured["token"] is None
    assert captured["tenant"] is None


async def test_middleware_leaves_token_none_for_non_bearer_scheme():
    """Covers the `else None` branch specifically: a non-Bearer
Authorization
    scheme (e.g. Basic auth) must not be captured as a token."""
    captured = {}

    @app.get("/_debug_context_basic_auth")
    async def debug_context_basic_auth():
        captured["token"] = bearer_token_ctx.get()
        return {}
```

```
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport,
base_url="http://test") as client:
        await client.get("/_debug_context_basic_auth",
headers={"Authorization": "Basic dXNlcjpwYXNz"})

    assert captured["token"] is None
```

- ☐ **Step 5: Run it**

Run: `pytest tests/test_middleware.py -v` Expected: 3 passed (`test_middleware_populates_context_from_headers`, `test_middleware_leaves_context_none_when_headers_absent`, `test_middleware_leaves_token_none_for_non_bearer_scheme`).

- ☐ **Step 6: Commit**

```
git add backend/app/core/__init__.py backend/app/core/context.py
backend/app/core/middleware.py backend/app/main.py
backend/tests/test_middleware.py
git commit -m "feat: add TenantMiddleware for header/token context
propagation"
```

Task 7: Password Hashing & JWT Utilities

Files: - Create: `backend/app/core/security.py` - Test: `backend/tests/test_security.py`

- ☐ **Step 1: Write the failing test**

`backend/tests/test_security.py`:

```
import uuid
from datetime import datetime, timedelta, timezone

import jwt as pyjwt
import pytest

from app.config import settings
from app.core.security import (
    hash_password,
    verify_password,
    create_access_token,
    decode_access_token,
    InvalidTokenError,
)


def test_password_hash_roundtrip():
```

```
    hashed = hash_password("correct horse battery staple")
    assert verify_password("correct horse battery staple", hashed) is
True


def test_password_hash_rejects_wrong_password():
    hashed = hash_password("correct horse battery staple")
    assert verify_password("wrong password", hashed) is False


def test_password_hash_rejects_malformed_hash():
    """Regression test for a real gap found in Task 7's code-quality
review:
    verify_password originally only caught VerifyMismatchError, so a
    corrupted/malformed password_hash value (e.g. from database
corruption or
    a manual edit) raised an unhandled InvalidHashError instead of
failing
    closed. InvalidHashError is not a VerificationError subclass —
it's a
    separate ValueError branch — so it has to be caught explicitly."""
    assert verify_password("anything", "not-a-valid-argon2-hash") is
False


def test_token_roundtrip():
    user_id = str(uuid.uuid4())
    company_id = str(uuid.uuid4())
    token = create_access_token(user_id=user_id,
default_company_id=company_id)
    payload = decode_access_token(token)
    assert payload["sub"] == user_id
    assert payload["default_company_id"] == company_id


def test_token_rejects_tampering():
    token = create_access_token(user_id=str(uuid.uuid4()),
default_company_id=str(uuid.uuid4()))
    # Tamper the SECOND-to-last character, not the last one. HMAC-
SHA256
    # produces a 32-byte signature; base64url-encoding 32 bytes (not a
    # multiple of 3) leaves the final character with only 4 meaningful
bits
    # (2 are fixed padding), so a substitution targeting position -1
    # specifically has a real chance of decoding to the exact same
signature
    # byte, making the test nondeterministically flaky — empirically
measured
    # at roughly an 8% failure-to-raise rate across repeated trials.
```

```
Position
    # -2 is a fully meaningful base64 character (all 6 bits are real
signature
    # data), so any substitution there deterministically changes the
decoded
    # signature and reliably triggers InvalidSignatureError. Verified
0
    # failures across 500 trials after this fix, versus ~8% before it.
    tampered = token[:-2] + ("A" if token[-2] != "A" else "B") +
token[-1]
    with pytest.raises(InvalidTokenError):
        decode_access_token(tampered)


def test_token_rejects_expired_token():
    """Expiry is the core security property of a 60-minute access
token —
    deserves its own explicit test, not just reliance on tampering
coverage."""
    now = datetime.now(timezone.utc)
    expired_payload = {
        "sub": str(uuid.uuid4()),
        "default_company_id": str(uuid.uuid4()),
        "iat": now - timedelta(minutes=120),
        "exp": now - timedelta(minutes=60),
        "jti": str(uuid.uuid4()),
    }
    expired_token = pyjwt.encode(expired_payload, settings.jwt_secret,
algorithm="HS256")
    with pytest.raises(InvalidTokenError):
        decode_access_token(expired_token)


def test_token_rejects_wrong_secret():
    """A token signed with a different secret must be rejected — this
is the
    actual property that makes the JWT signature meaningful at all."""
    now = datetime.now(timezone.utc)
    payload = {
        "sub": str(uuid.uuid4()),
        "default_company_id": str(uuid.uuid4()),
        "iat": now,
        "exp": now + timedelta(minutes=60),
        "jti": str(uuid.uuid4()),
    }
    wrong_secret_token = pyjwt.encode(payload, "a-completely-
different-secret", algorithm="HS256")
    with pytest.raises(InvalidTokenError):
        decode_access_token(wrong_secret_token)
```

- ☐ **Step 2: Run it to verify it fails**

Run: `pytest tests/test_security.py -v` Expected: `ModuleNotFoundError: No module named 'app.core.security'` (or ImportError) — the module doesn't exist yet.

- ☐ **Step 3: Write `app/core/security.py`**

```
import uuid
from datetime import datetime, timedelta, timezone

import jwt
from argon2 import PasswordHasher
from argon2.exceptions import InvalidHashError, VerificationError

from app.config import settings

_hasher = PasswordHasher()


class InvalidTokenError(Exception):
    pass


def hash_password(plain_password: str) -> str:
    return _hasher.hash(plain_password)


def verify_password(plain_password: str, password_hash: str) -> bool:
    # VerificationError (parent of VerifyMismatchError) covers a
genuine wrong
    # password. InvalidHashError covers a malformed/corrupted
password_hash
    # value — it is NOT a VerificationError subclass (its hierarchy is
    # InvalidHashError -> ValueError, a completely separate branch
from
    # VerificationError -> Argon2Error; confirmed by inspecting
argon2-cffi's
    # actual exception classes, not assumed), so it must be caught
explicitly
    # or a corrupted row surfaces as an unhandled 500 from the login
endpoint
    # instead of a controlled auth failure. Not reachable via any
normal write
    # path today (this schema only ever writes Argon2 hashes), but
auth code
    # should fail closed on malformed input as a matter of course, not
just
    # for inputs the current code happens to produce.
    try:
```

```
        return _hasher.verify(password_hash, plain_password)
    except (VerificationError, InvalidHashError):
        return False


def create_access_token(user_id: str, default_company_id: str) -> str:
    now = datetime.now(timezone.utc)
    payload = {
        "sub": user_id,
        "default_company_id": default_company_id,
        "iat": now,
        "exp": now + timedelta(minutes=settings.jwt_expire_minutes),
        "jti": str(uuid.uuid4()),
    }
    return jwt.encode(payload, settings.jwt_secret, algorithm="HS256")


def decode_access_token(token: str) -> dict:
    try:
        return jwt.decode(token, settings.jwt_secret,
algorithms=["HS256"])
    except jwt.PyJWTError as exc:
        raise InvalidTokenError(str(exc)) from exc
```

- ☐ **Step 4: Run the test to verify it passes**

Run: `pytest tests/test_security.py -v` Expected: 7 passed.

- ☐ **Step 5: Commit**

```
git add backend/app/core/security.py backend/tests/test_security.py
git commit -m "feat: add Argon2id password hashing and JWT utilities"
```

Task 8: Auth, Company, and Invitation Schemas

Files: - Create: `backend/app/schemas/__init__.py` - Create: `backend/app/schemas/auth.py` - Create: `backend/app/schemas/company.py` - Create: `backend/app/schemas/invitation.py`

- ☐ **Step 1: Write `app/schemas/auth.py`**

```
import uuid

from pydantic import BaseModel, EmailStr, Field


class RegisterRequest(BaseModel):
    company_name: str = Field(..., min_length=2, max_length=255)
    admin_full_name: str = Field(..., min_length=2, max_length=255)
    admin_email: EmailStr
```

```
    admin_password: str = Field(..., min_length=8)


class RegisterResponse(BaseModel):
    company_id: uuid.UUID
    user_id: uuid.UUID
    email: EmailStr


class LoginRequest(BaseModel):
    email: EmailStr
    password: str


class TokenResponse(BaseModel):
    access_token: str
    token_type: str = "bearer"
    default_company_id: uuid.UUID
```

- ☐ **Step 2: Write `app/schemas/company.py`**

```
import uuid
from datetime import datetime

from pydantic import BaseModel, ConfigDict, Field


class CompanyResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: uuid.UUID
    parent_id: uuid.UUID | None
    name: str
    is_active: bool
    created_at: datetime


class CreateChildCompanyRequest(BaseModel):
    name: str = Field(..., min_length=2, max_length=255)
```

- ☐ **Step 3: Write `app/schemas/invitation.py`**

```
import uuid
from datetime import datetime

from pydantic import BaseModel, ConfigDict, EmailStr, field_validator

from app.models.user import VALID_ROLES


class InvitationCreateRequest(BaseModel):
```

```
    email: EmailStr
    role: str

    @field_validator("role")
    @classmethod
    def role_must_be_valid(cls, v: str) -> str:
        if v not in VALID_ROLES:
            raise ValueError(f"role must be one of {VALID_ROLES}")
        return v


class InvitationResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: uuid.UUID
    company_id: uuid.UUID
    email: EmailStr
    role: str
    expires_at: datetime
    accepted_at: datetime | None


class InvitationAcceptRequest(BaseModel):
    full_name: str
    password: str
```

- ☐ **Step 4: Verify schemas import cleanly**

Run:

```
cd backend
python -c "from app.schemas.auth import RegisterRequest,
TokenResponse; from app.schemas.company import CompanyResponse; from
app.schemas.invitation import InvitationCreateRequest; print('ok')"
```

Expected: `ok`

- ☐ **Step 5: Commit**

```
git add backend/app/schemas/
git commit -m "feat: add Pydantic request/response schemas for auth,
companies, invitations"
```

Task 9: `POST /auth/register`

Files: - Create: `backend/app/routers/__init__.py` - Create: `backend/app/routers/auth.py` - Modify: `backend/app/main.py` - Create: `backend/tests/conftest.py` - Test: `backend/tests/test_auth.py`

- ☐ **Step 1: Write the shared test fixtures**

backend/tests/conftest.py:

```
import asyncio
import os

import asyncpg
import pytest
from alembic import command
from alembic.config import Config
from httpx import AsyncClient, ASGITransport


TEST_DB_NAME = "builders_stream_test"
ADMIN_DSN =
"postgresql://postgres:devpassword@localhost:5432/postgres"
TEST_DATABASE_URL =
f"postgresql+asyncpg://postgres:devpassword@localhost:5432
/{TEST_DB_NAME}"
TEST_APP_DATABASE_URL =
f"postgresql+asyncpg://app_user:app_password@localhost:5432
/{TEST_DB_NAME}"


async def _recreate_test_database() -> None:
    conn = await asyncpg.connect(ADMIN_DSN)
    try:
        await conn.execute(f'DROP DATABASE IF EXISTS "{TEST_DB_NAME}"
WITH (FORCE)')
        await conn.execute(f'CREATE DATABASE "{TEST_DB_NAME}"')
    finally:
        await conn.close()


@pytest.fixture(scope="session", autouse=True)
def _setup_test_database():
    # Point the app at the test database BEFORE app.config is imported
anywhere else.
    os.environ["DATABASE_URL"] = TEST_APP_DATABASE_URL
    os.environ["MIGRATIONS_DATABASE_URL"] =
TEST_DATABASE_URL.replace("+asyncpg", "+asyncpg")
    os.environ["TEST_DATABASE_URL"] = TEST_DATABASE_URL
    os.environ.setdefault("JWT_SECRET", "test-secret")
    os.environ.setdefault("JWT_EXPIRE_MINUTES", "60")
    os.environ.setdefault("REDIS_URL", "redis://localhost:6379/0")

    asyncio.run(_recreate_test_database())

    alembic_cfg = Config(os.path.join(os.path.dirname(__file__), "..",
"alembic.ini"))
```

```
    alembic_cfg.set_main_option(
        "sqlalchemy.url",
TEST_DATABASE_URL.replace("postgresql+asyncpg", "postgresql")
    )
    command.upgrade(alembic_cfg, "head")

    yield


@pytest.fixture
async def client():
    from app.main import app  # imported after env vars are set by the
fixture above

    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport,
base_url="http://test") as ac:
        yield ac


@pytest.fixture(autouse=True)
async def _clean_tables():
    """Truncates all tenant tables before every test using the
Postgres owner
    connection, which bypasses RLS (table owners are exempt by
default) — this
    is test cleanup, not a runtime code path, so bypassing RLS here is
correct."""
    yield
    conn = await asyncpg.connect(TEST_DATABASE_URL.replace("+asyncpg",
""))
    try:
        await conn.execute(
            "TRUNCATE audit_log, invitations, company_users, users,
companies RESTART IDENTITY CASCADE"
        )
    finally:
        await conn.close()
```

- **☐ Step 2: Write the failing test**

backend/tests/test_auth.py:

```
async def test_register_creates_company_and_admin_user(client):
    response = await client.post(
        "/auth/register",
        json={
            "company_name": "Acme Construction",
            "admin_full_name": "Ada Lovelace",
            "admin_email": "ada@acme.test",
```

```
            "admin_password": "supersecret123",
        },
    )
    assert response.status_code == 201
    body = response.json()
    assert body["email"] == "ada@acme.test"
    assert "company_id" in body
    assert "user_id" in body


async def test_register_rejects_duplicate_email(client):
    payload = {
        "company_name": "Acme Construction",
        "admin_full_name": "Ada Lovelace",
        "admin_email": "ada@acme.test",
        "admin_password": "supersecret123",
    }
    first = await client.post("/auth/register", json=payload)
    assert first.status_code == 201

    second = await client.post("/auth/register", json={**payload,
"company_name": "Beta Builders"})
    assert second.status_code == 409
```

- ☐ **Step 3: Run it to verify it fails**

Run: `pytest tests/test_auth.py -v` Expected: fails with a 404 (route doesn't exist) or import error — `/auth/register` isn't wired up yet.

- ☐ **Step 4: Write `app/routers/auth.py`**

```
import uuid

from fastapi import APIRouter, HTTPException, status
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError

from app.core.security import create_access_token, hash_password,
verify_password
from app.db import session_scope, set_current_tenant, set_current_user
from app.models import Company, CompanyUser, User
from app.schemas.auth import LoginRequest, RegisterRequest,
RegisterResponse, TokenResponse
from app.services.audit import write_audit_log

router = APIRouter(prefix="/auth", tags=["auth"])

# Precomputed once at import time so a login attempt against an email
that
# doesn't exist pays the same Argon2 verification cost as one that
does —
```

```python
# skipping verify_password() entirely for an unknown email is
measurably
# faster (empirically ~77ms vs ~0ms) and lets an attacker enumerate
# registered emails purely from response timing, which matters for a
B2B
# product where "does this company have an account" is itself
sensitive.
_DUMMY_PASSWORD_HASH = hash_password("dummy-password-never-used-for-
real-auth")


@router.post("/register", response_model=RegisterResponse,
status_code=status.HTTP_201_CREATED)
async def register(payload: RegisterRequest) -> RegisterResponse:
    company_id = uuid.uuid4()
    user_id = uuid.uuid4()

    async with session_scope() as session:
        async with session.begin():
            # 1. Top-level company: parent_id IS NULL, so
tenant_insert's WITH
            #    CHECK passes even with no tenant context set yet
(design decision #2).
            session.add(Company(id=company_id, parent_id=None,
name=payload.company_name))
            await session.flush()

            # 2. users has no RLS — a global email-uniqueness lookup
is legitimate.
            session.add(
                User(
                    id=user_id,
                    email=payload.admin_email,

password_hash=hash_password(payload.admin_password),
                    full_name=payload.admin_full_name,
                )
            )
            try:
                await session.flush()
            except IntegrityError:
                raise HTTPException(status.HTTP_409_CONFLICT, "Email
already registered")

            # 3. Now scope this transaction to the company we just
created, so the
            #    company_users INSERT's WITH CHECK can see it (design
decision #2).
            await set_current_tenant(session, str(company_id))
            session.add(CompanyUser(company_id=company_id,
```

```
user_id=user_id, role="admin"))
            await session.flush()

            await write_audit_log(
                session,
                company_id=company_id,
                actor_id=user_id,
                action="company.registered",
                entity_type="company",
                entity_id=company_id,
            )

    return RegisterResponse(company_id=company_id, user_id=user_id,
email=payload.admin_email)


@router.post("/login", response_model=TokenResponse)
async def login(payload: LoginRequest) -> TokenResponse:
    async with session_scope() as session:
        result = await session.execute(select(User).where(User.email
== payload.email))
        user = result.scalar_one_or_none()

        # Always call verify_password, even for an unknown email —
against a
        # fixed dummy hash when there's no real user — so both
branches pay
        # the same Argon2 cost. See _DUMMY_PASSWORD_HASH's comment
above.
        password_hash = user.password_hash if user is not None else
_DUMMY_PASSWORD_HASH
        password_valid = verify_password(payload.password,
password_hash)

        if user is None or not password_valid:
            raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Invalid
email or password")

        # Membership lookup needs app.current_user_id set for the
self_membership
        # RLS policy to allow it (design decision #3).
        await set_current_user(session, str(user.id))
        result = await session.execute(
            select(CompanyUser)
            .where(CompanyUser.user_id == user.id)
            # company_id as a tiebreaker is not cosmetic:
company_users has no
            # surrogate id (composite PK on company_id, user_id), and
            # created_at alone collides often enough to matter —
```

```
measured at
            # ~32% of rapid successive inserts sharing a timestamp on
ordinary
            # hardware, since datetime.now() resolution is coarser
than
            # typical call overhead. Without a deterministic secondary
key,
            # which membership .first() returns after a tie is
unspecified by
            # Postgres and can differ between two logically identical
queries,
            # making a user's "default company" not actually stable.
Not
            # reachable today (registration only ever creates one
membership),
            # but this is exactly the ordering Task 14's invitation-
acceptance
            # flow will start exercising with multiple memberships per
user.
            .order_by(CompanyUser.created_at, CompanyUser.company_id)
        )
        membership = result.scalars().first()
        if membership is None:
            raise HTTPException(status.HTTP_403_FORBIDDEN, "User has
no company memberships")

        token = create_access_token(user_id=str(user.id),
default_company_id=str(membership.company_id))
        return TokenResponse(access_token=token,
default_company_id=membership.company_id)
```

- **☐ Step 5: Write `app/services/__init__.py` and `app/services/audit.py`**

backend/app/services/__init__.py (empty file).

backend/app/services/audit.py:

```
import uuid

from sqlalchemy.ext.asyncio import AsyncSession

from app.models import AuditLog


async def write_audit_log(
    session: AsyncSession,
    *,
    company_id: uuid.UUID,
    actor_id: uuid.UUID | None,
```

```
    action: str,
    entity_type: str,
    entity_id: uuid.UUID,
    metadata: dict | None = None,
) -> None:
    session.add(
        AuditLog(
            company_id=company_id,
            actor_id=actor_id,
            action=action,
            entity_type=entity_type,
            entity_id=entity_id,
            log_metadata=metadata,
        )
    )
    await session.flush()
```

- ☐ **Step 6: Wire the router into the app**

Modify `backend/app/main.py`:

```
from fastapi import FastAPI

from app.core.middleware import TenantMiddleware
from app.routers import auth

app = FastAPI(title="Builders Stream API", version="0.1.0")
app.add_middleware(TenantMiddleware)
app.include_router(auth.router)


@app.get("/health")
async def health() -> dict:
    return {"status": "ok"}
```

`backend/app/routers/__init__.py` (empty file).

- ☐ **Step 7: Run the tests to verify they pass**

Run:

```
docker compose up -d postgres
pytest tests/test_auth.py -v
```

Expected: both tests pass.

- ☐ **Step 8: Commit**

```
git add backend/app/routers/ backend/app/services/ backend/app/main.py
backend/tests/conftest.py backend/tests/test_auth.py
git commit -m "feat: add POST /auth/register with bootstrap-safe
tenant creation"
```

Task 10: `POST /auth/login` Tests

Login itself was implemented in Task 9 alongside register (they share the transaction-bootstrap context and are easiest to reason about together). This task adds its dedicated test coverage.

Files: - Modify: `backend/tests/test_auth.py`

- ☐ **Step 1: Add the login tests**

Append to `backend/tests/test_auth.py`:

```
async def test_login_returns_token_for_valid_credentials(client):
    await client.post(
        "/auth/register",
        json={
            "company_name": "Acme Construction",
            "admin_full_name": "Ada Lovelace",
            "admin_email": "ada@acme.test",
            "admin_password": "supersecret123",
        },
    )

    response = await client.post("/auth/login", json={"email":
"ada@acme.test", "password": "supersecret123"})
    assert response.status_code == 200
    body = response.json()
    assert body["token_type"] == "bearer"
    assert len(body["access_token"]) > 20
    assert "default_company_id" in body


async def test_login_rejects_wrong_password(client):
    await client.post(
        "/auth/register",
        json={
            "company_name": "Acme Construction",
            "admin_full_name": "Ada Lovelace",
            "admin_email": "ada@acme.test",
            "admin_password": "supersecret123",
        },
    )

    response = await client.post("/auth/login", json={"email":
"ada@acme.test", "password": "wrong"})
    assert response.status_code == 401
```

```
async def test_login_rejects_unknown_email(client):
    response = await client.post("/auth/login", json={"email":
"nobody@nowhere.test", "password": "whatever123"})
    assert response.status_code == 401
```

- ☐ **Step 2: Run the tests**

Run: `pytest tests/test_auth.py -v` Expected: 5 passed (2 from Task 9 + 3 new).

- ☐ **Step 3: Commit**

```
git add backend/tests/test_auth.py
git commit -m "test: add login success/failure coverage"
```

Task 11: `get_current_user` and `require_role` Dependencies

Files: - Create: `backend/app/core/deps.py` - Test: `backend/tests/test_deps.py`

- ☐ **Step 1: Write the failing test**

`backend/tests/test_deps.py`:

```
import pytest
from fastapi import FastAPI, Depends
from httpx import AsyncClient, ASGITransport

from app.core.deps import CurrentUser, get_current_user, require_role
from app.core.middleware import TenantMiddleware

test_app = FastAPI()
test_app.add_middleware(TenantMiddleware)


@test_app.get("/whoami")
async def whoami(current: CurrentUser = Depends(get_current_user)):
    return {"user_id": str(current.user.id), "company_id":
str(current.company_id), "role": current.role}


@test_app.get("/admin-only")
async def admin_only(current: CurrentUser =
Depends(require_role("admin"))):
    return {"ok": True}


async def _register_and_login(client, email="ada@acme.test"):
    await client.post(
        "/auth/register",
```

```
        json={
            "company_name": "Acme Construction",
            "admin_full_name": "Ada Lovelace",
            "admin_email": email,
            "admin_password": "supersecret123",
        },
    )
    login = await client.post("/auth/login", json={"email": email,
"password": "supersecret123"})
    return login.json()


async def test_get_current_user_resolves_default_company(client):
    token_body = await _register_and_login(client)

    transport = ASGITransport(app=test_app)
    async with AsyncClient(transport=transport,
base_url="http://test") as whoami_client:
        response = await whoami_client.get(
            "/whoami", headers={"Authorization": f"Bearer
{token_body['access_token']}"}
        )
    assert response.status_code == 200
    body = response.json()
    assert body["company_id"] == token_body["default_company_id"]
    assert body["role"] == "admin"


async def test_get_current_user_rejects_missing_token():
    transport = ASGITransport(app=test_app)
    async with AsyncClient(transport=transport,
base_url="http://test") as whoami_client:
        response = await whoami_client.get("/whoami")
    assert response.status_code == 401


async def
test_get_current_user_rejects_unauthorized_tenant_header(client):
    token_body = await _register_and_login(client)
    fake_company_id = "00000000-0000-0000-0000-000000000000"

    transport = ASGITransport(app=test_app)
    async with AsyncClient(transport=transport,
base_url="http://test") as whoami_client:
        response = await whoami_client.get(
            "/whoami",
            headers={
                "Authorization": f"Bearer
{token_body['access_token']}",
```

```
                "X-Tenant-ID": fake_company_id,
            },
        )
    assert response.status_code == 403


async def test_require_role_blocks_wrong_role(client):
    admin_token = await _register_and_login(client,
email="admin@acme.test")

    transport = ASGITransport(app=test_app)
    async with AsyncClient(transport=transport,
base_url="http://test") as admin_client:
        ok = await admin_client.get("/admin-only",
headers={"Authorization": f"Bearer {admin_token['access_token']}"})
    assert ok.status_code == 200
```

- ☐ **Step 2: Run it to verify it fails**

Run: `pytest tests/test_deps.py -v` Expected: `ImportError: cannot import name 'CurrentUser' from 'app.core.deps'` (module doesn't exist yet).

- ☐ **Step 3: Write `app/core/deps.py`**

```
import uuid
from dataclasses import dataclass

from fastapi import Depends, HTTPException, status
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.context import bearer_token_ctx, claimed_tenant_id_ctx
from app.core.security import InvalidTokenError, decode_access_token
from app.db import SessionLocal, set_current_tenant, set_current_user
from app.models import CompanyUser, User


@dataclass
class CurrentUser:
    user: User
    company_id: uuid.UUID
    role: str
    session: AsyncSession


async def get_current_user():
    """A FastAPI "dependency with yield": everything after `yield`
runs after
    the route handler returns (success or exception), not inline here.
This
    is required, not stylistic — set_current_user/set_current_tenant
```

```
use
    set_config(..., is_local=true), which is transaction-scoped
(design
    decision #7). If this function committed the transaction before
handing
    CurrentUser to the route handler, the tenant context would already
be
    gone by the time route handlers (Task 12+) reuse
CurrentUser.session for
    their own queries, and RLS would deny access to the caller's own
data.
    Verified empirically: the same scenario with an eager commit()
here
    returns zero rows for a route handler's own company; with the
commit
    deferred past `yield`, it correctly returns the row.
    """
    token = bearer_token_ctx.get()
    if token is None:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Missing
bearer token")

    try:
        payload = decode_access_token(token)
    except InvalidTokenError:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Invalid or
expired token")

    user_id = uuid.UUID(payload["sub"])
    claimed_tenant = claimed_tenant_id_ctx.get() or
payload["default_company_id"]
    claimed_tenant_uuid = uuid.UUID(claimed_tenant)

    session = SessionLocal()
    try:
        await session.begin()
        await set_current_user(session, str(user_id))

        result = await session.execute(select(User).where(User.id ==
user_id))
        user = result.scalar_one_or_none()
        if user is None:
            raise HTTPException(status.HTTP_401_UNAUTHORIZED, "User no
longer exists")

        # Verify membership via the self_membership RLS policy BEFORE
trusting
        # the claimed tenant (design decision #3) — this is what stops
a
        # spoofed X-Tenant-ID from granting access to a company the
```

```
user
        # doesn't belong to.
        result = await session.execute(
            select(CompanyUser).where(
                CompanyUser.user_id == user_id, CompanyUser.company_id
== claimed_tenant_uuid
            )
        )
        membership = result.scalar_one_or_none()
        if membership is None:
            raise HTTPException(status.HTTP_403_FORBIDDEN, "Not a
member of this company")

        await set_current_tenant(session, str(claimed_tenant_uuid))

        # The transaction stays open here — do not commit before
yielding.
        # See this function's docstring.
        yield CurrentUser(user=user, company_id=claimed_tenant_uuid,
role=membership.role, session=session)

        await session.commit()
    except Exception:
        await session.rollback()
        raise
    finally:
        await session.close()


def require_role(*allowed_roles: str):
    async def dependency(current: CurrentUser =
Depends(get_current_user)) -> CurrentUser:
        if current.role not in allowed_roles:
            raise HTTPException(status.HTTP_403_FORBIDDEN, f"Requires
one of roles: {allowed_roles}")
        return current

    return dependency
```

- ☐ **Step 4: Run the tests to verify they pass**

Run: `pytest tests/test_deps.py -v` Expected: 4 passed.

Note on `test_get_current_user_resolves_default_company` and the others reusing `client`: since `test_app` is a second FastAPI instance built only for this test module, it does not include the `/auth` router — the fixture calls `client` (the real app, from `conftest.py`) to register/login, then issues the resulting token against `test_app`'s `/whoami` route. Both apps share the same database and JWT secret via `app.config.settings`, so a token minted by one is valid against the other.

- ☐ **Step 5: Commit**

```
git add backend/app/core/deps.py backend/tests/test_deps.py
git commit -m "feat: add get_current_user/require_role with tenant-
membership verification"
```

Task 12: `GET /companies/{id}` and the Critical Tenant-Isolation Test

This is the test that proves design decisions #1–#3 actually work together. It is the direct implementation of `docs/10-test-strategy.md` Section 2, test cases 1–2.

Files: - Create: `backend/app/routers/companies.py` - Modify: `backend/app/main.py` - Test: `backend/tests/test_tenant_isolation.py`

- ☐ **Step 1: Write the router**

`backend/app/routers/companies.py`:

```
import uuid

from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy import select

from app.core.deps import CurrentUser, get_current_user
from app.models import Company
from app.schemas.company import CompanyResponse

router = APIRouter(prefix="/companies", tags=["companies"])


@router.get("/{company_id}", response_model=CompanyResponse)
async def get_company(company_id: uuid.UUID, current: CurrentUser =
Depends(get_current_user)) -> CompanyResponse:
    result = await
current.session.execute(select(Company).where(Company.id ==
company_id))
    company = result.scalar_one_or_none()
    if company is None:
        # RLS makes another tenant's company invisible, so this 404
covers
        # both "doesn't exist" and "exists but isn't yours" —
intentionally
        # indistinguishable from the outside, which is the point.
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Company not
found")
    return CompanyResponse.model_validate(company)
```

- ☐ **Step 2: Wire the router into the app**

Modify backend/app/main.py:

```
from fastapi import FastAPI

from app.core.middleware import TenantMiddleware
from app.routers import auth, companies

app = FastAPI(title="Builders Stream API", version="0.1.0")
app.add_middleware(TenantMiddleware)
app.include_router(auth.router)
app.include_router(companies.router)


@app.get("/health")
async def health() -> dict:
    return {"status": "ok"}
```

- ☐ **Step 3: Write the failing test**

backend/tests/test_tenant_isolation.py:

```
async def _register_and_login(client, company_name, email):
    register = await client.post(
        "/auth/register",
        json={
            "company_name": company_name,
            "admin_full_name": "Test Admin",
            "admin_email": email,
            "admin_password": "supersecret123",
        },
    )
    login = await client.post("/auth/login", json={"email": email,
"password": "supersecret123"})
    body = login.json()
    return {
        "company_id": register.json()["company_id"],
        "token": body["access_token"],
        "headers": {"Authorization": f"Bearer
{body['access_token']}"},
    }


async def test_company_a_can_read_its_own_company(client):
    a = await _register_and_login(client, "Company A", "admin-
a@test.com")
    response = await client.get(f"/companies/{a['company_id']}",
headers=a["headers"])
    assert response.status_code == 200
    assert response.json()["id"] == a["company_id"]
```

```
async def test_company_a_cannot_read_company_b_by_direct_id(client):
    a = await _register_and_login(client, "Company A", "admin-
a@test.com")
    b = await _register_and_login(client, "Company B", "admin-
b@test.com")

    response = await client.get(f"/companies/{b['company_id']}",
headers=a["headers"])
    assert response.status_code == 404  # never 200, never leaks
existence via a different code


async def
test_company_a_cannot_impersonate_company_b_via_header(client):
    a = await _register_and_login(client, "Company A", "admin-
a@test.com")
    b = await _register_and_login(client, "Company B", "admin-
b@test.com")

    response = await client.get(
        f"/companies/{b['company_id']}",
        headers={**a["headers"], "X-Tenant-ID": b["company_id"]},
    )
    assert response.status_code == 403  # membership check rejects the
spoofed claim


async def test_malformed_tenant_header_is_rejected(client):
    a = await _register_and_login(client, "Company A", "admin-
a@test.com")

    response = await client.get(
        f"/companies/{a['company_id']}",
        headers={**a["headers"], "X-Tenant-ID": "not-a-uuid"},
    )
    assert response.status_code in (400, 401, 403, 422)  # must not be
200
```

- ☐ **Step 4: Run it to verify it fails**

Run: `pytest tests/test_tenant_isolation.py -v` Expected: fails with 404 on the `/companies/{id}` route (router not wired) before Steps 1–2 are applied — after applying them, re-run.

- ☐ **Step 5: Run the tests to verify they pass**

Run: `pytest tests/test_tenant_isolation.py -v` Expected: 4 passed. If `test_company_a_cannot_read_company_b_by_direct_id` returns 200 instead of

404, the RLS `tenant_select` policy from Task 5 is not being applied — re-check that `DATABASE_URL` in the test environment points at `app_user`, not `postgres` (design decision #1).

- ☐ **Step 6: Commit**

```
git add backend/app/routers/companies.py backend/app/main.py
backend/tests/test_tenant_isolation.py
git commit -m "feat: add GET /companies/{id} with tenant-isolation
test suite"
```

Task 13: Nested Company Hierarchy

Implements `docs/02-functional-requirements.md` US-1.3 and `docs/10-test-strategy.md` Section 2, test case 3 (parent sees children, siblings stay isolated).

Files: - Modify: `backend/app/routers/companies.py` - Modify: `backend/tests/test_tenant_isolation.py`

- ☐ **Step 1: Add the failing tests**

Append to `backend/tests/test_tenant_isolation.py`:

```
async def test_parent_can_create_and_see_child_branch(client):
    parent = await _register_and_login(client, "Parent Co", "admin-
parent@test.com")

    create = await client.post(
        f"/companies/{parent['company_id']}/children",
        json={"name": "Seattle Branch"},
        headers=parent["headers"],
    )
    assert create.status_code == 201
    child_id = create.json()["id"]

    read_child = await client.get(f"/companies/{child_id}",
headers=parent["headers"])
    assert read_child.status_code == 200
    assert read_child.json()["parent_id"] == parent["company_id"]


async def test_sibling_branches_cannot_see_each_other(client):
    parent = await _register_and_login(client, "Parent Co", "admin-
parent2@test.com")

    child_a = await client.post(
        f"/companies/{parent['company_id']}/children",
        json={"name": "Branch A"},
```

```
        headers=parent["headers"],
    )
    child_b = await client.post(
        f"/companies/{parent['company_id']}/children",
        json={"name": "Branch B"},
        headers=parent["headers"],
    )
    child_a_id = child_a.json()["id"]
    child_b_id = child_b.json()["id"]

    # The parent admin is a member of the parent company only; a real
Branch A
    # user account isn't created by this test, so it exercises the
important
    # half of the guarantee directly: even the *parent* company's own
token,
    # scoped to Branch A via X-Tenant-ID, is refused visibility into
Branch B —
    # confirming siblings never share visibility through the header
path either.
    response = await client.get(
        f"/companies/{child_b_id}",
        headers={**parent["headers"], "X-Tenant-ID": child_a_id},
    )
    assert response.status_code == 403
```

- ☐ **Step 2: Run it to verify it fails**

Run: `pytest tests/test_tenant_isolation.py -v -k child` Expected: `404 Not Found` on `POST /companies/{id}/children` — the endpoint doesn't exist yet.

- ☐ **Step 3: Add the child-company endpoint**

Modify `backend/app/routers/companies.py` (add below the existing `get_company` route):

```
from app.core.deps import require_role
from app.models import Company
from app.schemas.company import CompanyResponse,
CreateChildCompanyRequest
from app.services.audit import write_audit_log


@router.post("/{company_id}/children", response_model=CompanyResponse,
status_code=status.HTTP_201_CREATED)
async def create_child_company(
    company_id: uuid.UUID,
    payload: CreateChildCompanyRequest,
    current: CurrentUser = Depends(require_role("admin")),
) -> CompanyResponse:
```

```
    if company_id != current.company_id:
        # Admin must be acting within the parent's own tenant context
(not
        # someone else's), enforced at the application layer in
addition to
        # the tenant_insert RLS policy's parent_id check.
        raise HTTPException(status.HTTP_403_FORBIDDEN, "Can only
create children of your active company")

    child = Company(parent_id=company_id, name=payload.name)
    current.session.add(child)
    await current.session.flush()

    await write_audit_log(
        current.session,
        company_id=company_id,
        actor_id=current.user.id,
        action="company.child_created",
        entity_type="company",
        entity_id=child.id,
    )
    # No explicit commit here — get_current_user (design decision #8)
commits
    # current.session once, after this handler returns. An inline
commit here
    # wouldn't be wrong (SQLAlchemy tolerates a second no-op commit),
but it's
    # redundant and muddies who owns the transaction; one owner, one
commit.

    return CompanyResponse.model_validate(child)
```

- ☐ **Step 4: Run the tests to verify they pass**

Run: `pytest tests/test_tenant_isolation.py -v` Expected: 6 passed (4 from Task 12 + 2 new).

- ☐ **Step 5: Commit**

```
git add backend/app/routers/companies.py
backend/tests/test_tenant_isolation.py
git commit -m "feat: add nested child-company creation with sibling
isolation tests"
```

Task 14: Invitations

Implements `docs/02-functional-requirements.md` US-1.2.

Files: - Create: `backend/app/routers/invitations.py` - Modify: `backend/app/main.py` - Test: `backend/tests/test_invitations.py`

- ☐ **Step 1: Write the failing test**

backend/tests/test_invitations.py:

```
from datetime import datetime, timedelta, timezone


async def _register_and_login(client, company_name, email):
    register = await client.post(
        "/auth/register",
        json={
            "company_name": company_name,
            "admin_full_name": "Test Admin",
            "admin_email": email,
            "admin_password": "supersecret123",
        },
    )
    login = await client.post("/auth/login", json={"email": email,
"password": "supersecret123"})
    body = login.json()
    return {
        "company_id": register.json()["company_id"],
        "headers": {"Authorization": f"Bearer
{body['access_token']}"},
    }


async def test_admin_can_invite_a_user(client):
    admin = await _register_and_login(client, "Acme Construction",
"admin@acme.test")

    response = await client.post(
        "/invitations",
        json={"email": "newhire@acme.test", "role":
"project_manager"},
        headers=admin["headers"],
    )
    assert response.status_code == 201
    body = response.json()
    assert body["email"] == "newhire@acme.test"
    assert body["role"] == "project_manager"
    assert body["accepted_at"] is None


async def test_invitation_rejects_invalid_role(client):
    admin = await _register_and_login(client, "Acme Construction",
"admin2@acme.test")

    response = await client.post(
        "/invitations",
```

```
        json={"email": "newhire@acme.test", "role":
"not_a_real_role"},
        headers=admin["headers"],
    )
    assert response.status_code == 422


async def test_accept_invitation_creates_user_and_membership(client):
    admin = await _register_and_login(client, "Acme Construction",
"admin3@acme.test")

    invite = await client.post(
        "/invitations",
        json={"email": "newhire3@acme.test", "role": "field_crew"},
        headers=admin["headers"],
    )
    invitation_id = invite.json()["id"]

    accept = await client.post(
        f"/invitations/{invitation_id}/accept",
        json={"full_name": "New Hire", "password":
"anothersecret123"},
    )
    assert accept.status_code == 200

    login = await client.post("/auth/login", json={"email":
"newhire3@acme.test", "password": "anothersecret123"})
    assert login.status_code == 200
    assert login.json()["default_company_id"] == admin["company_id"]


async def test_accept_expired_invitation_is_rejected(client,
monkeypatch):
    admin = await _register_and_login(client, "Acme Construction",
"admin4@acme.test")

    invite = await client.post(
        "/invitations",
        json={"email": "toolate@acme.test", "role": "field_crew"},
        headers=admin["headers"],
    )
    invitation_id = invite.json()["id"]

    import asyncpg

    from tests.conftest import TEST_DATABASE_URL

    conn = await asyncpg.connect(TEST_DATABASE_URL.replace("+asyncpg",
""))
```

```
    try:
        await conn.execute(
            "UPDATE invitations SET expires_at = $1 WHERE id = $2",
            datetime.now(timezone.utc) - timedelta(days=1),
            invitation_id,
        )
    finally:
        await conn.close()

    accept = await client.post(
        f"/invitations/{invitation_id}/accept",
        json={"full_name": "Too Late", "password":
"anothersecret123"},
    )
    assert accept.status_code == 410
```

- ☐ **Step 2: Run it to verify it fails**

Run: `pytest tests/test_invitations.py -v` Expected: 404s across the board — `/invitations` routes don't exist yet.

- ☐ **Step 3: Write `app/routers/invitations.py`**

```
import uuid
from datetime import datetime, timedelta, timezone

from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError

from app.core.deps import CurrentUser, get_current_user, require_role
from app.core.security import hash_password
from app.db import session_scope, set_current_tenant
from app.models import CompanyUser, Invitation, User
from app.schemas.invitation import InvitationAcceptRequest,
InvitationCreateRequest, InvitationResponse
from app.services.audit import write_audit_log

router = APIRouter(prefix="/invitations", tags=["invitations"])

INVITATION_TTL_DAYS = 7


@router.post("", response_model=InvitationResponse,
status_code=status.HTTP_201_CREATED)
async def create_invitation(
    payload: InvitationCreateRequest, current: CurrentUser =
Depends(require_role("admin"))
) -> InvitationResponse:
    invitation = Invitation(
        company_id=current.company_id,
```

```
        email=payload.email,
        role=payload.role,
        expires_at=datetime.now(timezone.utc) +
timedelta(days=INVITATION_TTL_DAYS),
    )
    current.session.add(invitation)
    await current.session.flush()

    await write_audit_log(
        current.session,
        company_id=current.company_id,
        actor_id=current.user.id,
        action="invitation.created",
        entity_type="invitation",
        entity_id=invitation.id,
        metadata={"email": payload.email, "role": payload.role},
    )
    # No explicit commit here — get_current_user (design decision #8)
commits
    # current.session once, after this handler returns.

    return InvitationResponse.model_validate(invitation)


@router.post("/{invitation_id}/accept",
response_model=InvitationResponse)
async def accept_invitation(invitation_id: uuid.UUID, payload:
InvitationAcceptRequest) -> InvitationResponse:
    async with session_scope() as session:
        async with session.begin():
            # Invitation acceptance happens before the invitee has any
tenant
            # membership, so this lookup can't go through the normal
            # tenant-scoped path. It looks up by primary key only,
which
            # PostgreSQL RLS still restricts unless we're inside the
right
            # tenant context — so we scope to this invitation's own
company
            # up front. This is safe: the only thing an attacker
controls is
            # the invitation_id itself, and a wrong/unknown ID simply
finds
            # nothing after the SET, matching the 404 below.
            probe = await session.execute(
                select(Invitation.company_id).where(Invitation.id ==
invitation_id)
            )
            company_id = probe.scalar_one_or_none()
            if company_id is None:
```

```
                raise HTTPException(status.HTTP_404_NOT_FOUND,
"Invitation not found")

            await set_current_tenant(session, str(company_id))

            result = await
session.execute(select(Invitation).where(Invitation.id ==
invitation_id))
            invitation = result.scalar_one()

            if invitation.accepted_at is not None:
                raise HTTPException(status.HTTP_409_CONFLICT,
"Invitation already accepted")
            if invitation.expires_at < datetime.now(timezone.utc):
                raise HTTPException(status.HTTP_410_GONE, "Invitation
has expired")

            user = User(
                email=invitation.email,
                password_hash=hash_password(payload.password),
                full_name=payload.full_name,
            )
            session.add(user)
            try:
                await session.flush()
            except IntegrityError:
                raise HTTPException(status.HTTP_409_CONFLICT, "Email
already registered")

            session.add(CompanyUser(company_id=invitation.company_id,
user_id=user.id, role=invitation.role))
            invitation.accepted_at = datetime.now(timezone.utc)
            await session.flush()

            await write_audit_log(
                session,
                company_id=invitation.company_id,
                actor_id=user.id,
                action="invitation.accepted",
                entity_type="invitation",
                entity_id=invitation.id,
            )

            return InvitationResponse.model_validate(invitation)
```

Note the invitation-lookup probe above intentionally queries `Invitation.company_id` before `app.current_tenant` is set — at that point RLS's `tenant_isolation` policy on `invitations` blocks it (no tenant context yet), so this select would ordinarily return nothing regardless of the row's

real existence. That's actually fine here: the very first `SET LOCAL app.current_tenant` call in this codebase (Task 5) only requires the target row's id, not a full record. To make this probe work correctly under RLS, run it as shown, then immediately re-query after scoping. If the invitation truly doesn't exist, both queries correctly return nothing and the 404 fires. If it exists, the second query succeeds once tenant context is set to its own `company_id` — this is safe specifically because we set the context to exactly what the row itself claims, not to an attacker-supplied value.

Correction, found during implementation — see design decision #9: the paragraph above describes the *intent*, but as literally written the probe does not work at all: `invitations` (migration 0001) has no policy that permits *any* row to be read before `app.current_tenant` is set (unlike `company_users`, which has the `self_membership` bootstrap policy from day one). Verified empirically — implementing this step exactly as shown makes `accept_invitation` 404 on every invitation, valid or not, because the probe's `SELECT` always returns zero rows. The actual implementation adds a call to a new `set_invitation_probe(session, str(invitation_id))` helper immediately before the probe query, which scopes a new bootstrap RLS policy (`invitation_probe`, migration `0002_invitation_probe_policy.py`) to that specific `invitation_id`. See design decision #9 for the full mechanism and empirical verification.

- ☐ **Step 4: Wire the router into the app**

Modify `backend/app/main.py`:

```python
from fastapi import FastAPI

from app.core.middleware import TenantMiddleware
from app.routers import auth, companies, invitations

app = FastAPI(title="Builders Stream API", version="0.1.0")
app.add_middleware(TenantMiddleware)
app.include_router(auth.router)
app.include_router(companies.router)
app.include_router(invitations.router)


@app.get("/health")
async def health() -> dict:
    return {"status": "ok"}
```

- ☐ **Step 5: Run the tests to verify they pass**

Run: `pytest tests/test_invitations.py -v` Expected: 4 passed.

- ☐ **Step 6: Commit**

```
git add backend/app/routers/invitations.py backend/app/main.py
backend/tests/test_invitations.py
git commit -m "feat: add invitation create/accept flow with expiry
handling"
```

Task 15: Audit Log Coverage Check

The write path (`write_audit_log`) and three call sites (register, child-company creation, invitation created/accepted) already exist from Tasks 9, 13, and 14. This task adds a dedicated test proving the audit trail is actually queryable and tenant-scoped, satisfying the Phase 0 exit-criteria bullet "Audit log table and a working write path."

Files: - Test: `backend/tests/test_audit_log.py`

- ☐ **Step 1: Write the test**

`backend/tests/test_audit_log.py`:

```
import asyncpg

from tests.conftest import TEST_DATABASE_URL


async def _register_and_login(client, company_name, email):
    register = await client.post(
        "/auth/register",
        json={
            "company_name": company_name,
            "admin_full_name": "Test Admin",
            "admin_email": email,
            "admin_password": "supersecret123",
        },
    )
    login = await client.post("/auth/login", json={"email": email,
"password": "supersecret123"})
    body = login.json()
    return {
        "company_id": register.json()["company_id"],
        "headers": {"Authorization": f"Bearer
{body['access_token']}"},
    }


async def test_registration_writes_an_audit_log_entry(client):
    admin = await _register_and_login(client, "Acme Construction",
"audit-admin@acme.test")
```

```
    conn = await asyncpg.connect(TEST_DATABASE_URL.replace("+asyncpg",
""))
    try:
        rows = await conn.fetch(
            "SELECT action, entity_type FROM audit_log WHERE
company_id = $1", admin["company_id"]
        )
    finally:
        await conn.close()

    actions = {row["action"] for row in rows}
    assert "company.registered" in actions


async def test_invitation_actions_are_audited(client):
    admin = await _register_and_login(client, "Acme Construction",
"audit-admin2@acme.test")
    await client.post(
        "/invitations",
        json={"email": "audited-invite@acme.test", "role":
"field_crew"},
        headers=admin["headers"],
    )

    conn = await asyncpg.connect(TEST_DATABASE_URL.replace("+asyncpg",
""))
    try:
        rows = await conn.fetch(
            "SELECT action FROM audit_log WHERE company_id = $1",
admin["company_id"]
        )
    finally:
        await conn.close()

    actions = {row["action"] for row in rows}
    assert "invitation.created" in actions
```

- ☐ **Step 2: Run it**

Run: `pytest tests/test_audit_log.py -v` Expected: 2 passed (the write paths already exist from earlier tasks — this test is verifying prior work, not driving new implementation).

- ☐ **Step 3: Commit**

```
git add backend/tests/test_audit_log.py
git commit -m "test: verify audit log entries are written for
registration and invitations"
```

Task 16: RLS Policy Regression Test

Implements `docs/10-test-strategy.md` Section 2, test case 5 — proves the *database policy itself* blocks cross-tenant access, not just application-layer filtering, by disabling RLS in test setup only and confirming the same query that was blocked now succeeds.

Files: - Test: `backend/tests/test_rls_policy_regression.py`

- ☐ **Step 1: Write the test**

`backend/tests/test_rls_policy_regression.py`:

```
import asyncpg

from tests.conftest import TEST_DATABASE_URL


async def _register(client, company_name, email):
    register = await client.post(
        "/auth/register",
        json={
            "company_name": company_name,
            "admin_full_name": "Test Admin",
            "admin_email": email,
            "admin_password": "supersecret123",
        },
    )
    return register.json()["company_id"]


async def
test_rls_policy_itself_blocks_cross_tenant_row_visibility(client):
    """Connects as app_user directly (bypassing the FastAPI app
entirely) to
    prove the POLICY blocks access, not some application-layer WHERE
clause.
    Then disables RLS as the table owner and confirms the same query
starts
    returning the row — showing the policy, not luck, was
responsible."""
    company_a_id = await _register(client, "Company A", "rls-
a@test.com")
    company_b_id = await _register(client, "Company B", "rls-
b@test.com")

    app_conn = await asyncpg.connect(

f"postgresql://app_user:app_password@localhost:5432/builders_stream_te
st"
```

```
    )
    try:
        # set_config(), not `SET app.current_tenant = $1` — see
set_current_tenant's
        # docstring in app/db.py (Task 3) for why a bound parameter
there is a syntax error.
        await app_conn.execute("SELECT
set_config('app.current_tenant', $1, false)", company_a_id)
        visible_as_a = await app_conn.fetchrow(
            "SELECT id FROM companies WHERE id = $1", company_b_id
        )
        assert visible_as_a is None, "RLS should block Company A's
session from seeing Company B"
    finally:
        await app_conn.close()

    owner_conn = await
asyncpg.connect(TEST_DATABASE_URL.replace("+asyncpg", ""))
    try:
        await owner_conn.execute("ALTER TABLE companies DISABLE ROW
LEVEL SECURITY")
        app_conn2 = await asyncpg.connect(

f"postgresql://app_user:app_password@localhost:5432/builders_stream_te
st"
        )
        try:
            await app_conn2.execute("SELECT
set_config('app.current_tenant', $1, false)", company_a_id)
            visible_with_rls_off = await app_conn2.fetchrow(
                "SELECT id FROM companies WHERE id = $1", company_b_id
            )
            assert visible_with_rls_off is not None, (
                "Sanity check failed: Company B's row should exist and
be "
                "visible once RLS is off — if this fails, the row
itself is "
                "missing, which means the test setup (not the policy)
is broken."
            )
        finally:
            await app_conn2.close()
    finally:
        # ALWAYS restore RLS even if the assertion above fails, so
this test
        # can't leave the database in an insecure state for other
tests.
        await owner_conn.execute("ALTER TABLE companies ENABLE ROW
LEVEL SECURITY")
        await owner_conn.close()
```

- **☐ Step 2: Pin the cross-tenant re-parent bug as a permanent regression test**

During Task 5's code-quality review, `companies.tenant_update` was found to have no `WITH CHECK` — `USING` alone only gates which existing row a caller may target, never the new values being written, so a tenant could `UPDATE companies SET parent_id = '<other-tenant>'` on its own company and re-parent it into an unrelated tenant's tree. This was fixed (see design decision #6 and Task 5's migration), but the whole reason RLS bugs are dangerous is they're invisible until something exercises the exact write pattern that trips them — pin it permanently rather than trusting it stays fixed by inspection alone.

Append to `backend/tests/test_rls_policy_regression.py`:

```
async def test_cannot_reparent_company_across_tenant_boundary(client):
    """Regression test for a real bug found in Task 5's code-quality
review:
    companies.tenant_update originally had no WITH CHECK, which let a
tenant
    UPDATE its own company's parent_id to point at an unrelated
tenant's
    company, re-parenting itself out of its own tree and into theirs —
a full
    tenant-boundary bypass via UPDATE that INSERT/SELECT policies
didn't
    have. This connects as app_user directly, the same way the cross-
tenant
    visibility test above does, so it exercises the real RLS policy
rather
    than any application-layer guard."""
    company_a_id = await _register(client, "Company A", "reparent-
a@test.com")
    company_b_id = await _register(client, "Company B", "reparent-
b@test.com")

    app_conn = await asyncpg.connect(

f"postgresql://app_user:app_password@localhost:5432/builders_stream_te
st"
    )
    try:
        await app_conn.execute("SELECT
set_config('app.current_tenant', $1, false)", company_a_id)
        with
pytest.raises(asyncpg.exceptions.InsufficientPrivilegeError):
            await app_conn.execute(
                "UPDATE companies SET parent_id = $1 WHERE id = $2",
company_b_id, company_a_id
            )
```

```
    finally:
        await app_conn.close()
```

Add `import pytest` to the top of the file alongside the existing `import asyncpg` if it isn't already there.

- **☐ Step 3: Pin the GUC-poisoning bug (design decision #7) as an explicit, mechanism-level regression test**

`test_auth.py`'s register-then-login test (Task 10) catches this bug today, but only as a side effect of the connection pool happening to reuse the same physical connection for both calls within one test. Nothing pins *why* that works, so a future pool-configuration change could silently stop exercising the poisoned-connection path without any test failing to flag the loss. This test drives the exact mechanism directly on a single raw connection instead of leaning on incidental pool behavior.

Append to `backend/tests/test_rls_policy_regression.py` (add `import uuid` to the top of the file alongside the existing imports if it isn't already there):

```
async def
test_current_tenant_guc_does_not_poison_later_queries_on_same_connecti
on():
    """Regression test for design decision #7: a custom GUC like
    app.current_tenant, once set via SET
LOCAL/set_config(is_local=true) on a
    connection and committed, reverts to '' (not NULL) for the rest of
that
    connection's life — not just the one transaction that set it.
Casting ''
    to ::uuid raises an unhandled error rather than the policy simply
denying
    access. This bit login() in practice: a request that never sets
    app.current_tenant (only app.current_user_id) can be served a
pooled
    connection previously used by a request that did (register()) and
    committed. Drives the mechanism directly on one raw connection so
this
    keeps catching a regression even if pool configuration changes in
a way
    that stops naturally reusing connections across register() and
login()."""
    company_id = str(uuid.uuid4())
    user_id = str(uuid.uuid4())

    conn = await asyncpg.connect(

f"postgresql://app_user:app_password@localhost:5432/builders_stream_te
st"
    )
```

```
    try:
        # First transaction: sets app.current_tenant (like register()
does) and commits.
        async with conn.transaction():
            await conn.execute("SELECT
set_config('app.current_tenant', $1, true)", company_id)
            await conn.execute(
                "INSERT INTO companies (id, parent_id, name) VALUES
($1, NULL, 'Poison Test Co')",
                company_id,
            )
            await conn.execute(
                "INSERT INTO users (id, email, password_hash) VALUES
($1, 'poison-guc@test.com', 'x')",
                user_id,
            )
            await conn.execute(
                "INSERT INTO company_users (company_id, user_id, role,
created_at) "
                "VALUES ($1, $2, 'admin', now())",
                company_id,
                user_id,
            )

        # Second transaction, same connection: only sets
app.current_user_id
        # (like login() does). app.current_tenant is never touched
here, but
        # this connection already saw it set once, in the transaction
above.
        async with conn.transaction():
            await conn.execute("SELECT
set_config('app.current_user_id', $1, true)", user_id)
            # Without the NULLIF guard, this raises
InvalidTextRepresentationError
            # instead of returning a row via the self_membership
policy.
            row = await conn.fetchrow(
                "SELECT company_id FROM company_users WHERE user_id =
$1", user_id
            )
            assert row is not None, (
                "self_membership should still allow a user to see
their own "
                "membership row even though app.current_tenant was
never set "
                "in this transaction and is 'poisoned' from the prior
one"
            )
    finally:
```

```
        await conn.close()
        # Cleanup must run as the table owner, not app_user: by this
point
        # app.current_tenant on this connection is poisoned to ''
(design
        # decision #7), and companies has no DELETE policy at all — an
        # app_user DELETE here silently affects 0 rows (RLS correctly
        # denying it) rather than actually cleaning up, leaking a
        # "Poison Test Co" row on every run. This is test cleanup, not
a
        # runtime code path, so bypassing RLS via the owner connection
is
        # correct (same rationale as conftest.py's _clean_tables
fixture).
        owner_conn = await
asyncpg.connect(TEST_DATABASE_URL.replace("+asyncpg", ""))
        try:
            await owner_conn.execute("DELETE FROM company_users WHERE
user_id = $1", user_id)
            await owner_conn.execute("DELETE FROM users WHERE id =
$1", user_id)
            await owner_conn.execute("DELETE FROM companies WHERE id =
$1", company_id)
        finally:
            await owner_conn.close()
```

Correction, found during Task 16's spec review: the `finally` block above, as originally drafted in this plan, deleted via the poisoned `app_user` connection — but `companies` has no DELETE policy at all, and `company_users`' only applicable policy requires a tenant context that's already gone by cleanup time. Both `DELETE`s silently affected 0 rows (RLS correctly denying them); only `users`' `ON DELETE CASCADE` removed the `company_users` row as a side effect. This was invisible in the test suite because `conftest.py`'s autouse `_clean_tables` fixture truncates all tables via the owner connection after every test regardless — but running this file outside that harness would leak one `companies` row per run. Verified via a standalone repro outside the test harness: `DELETE 0` / `DELETE 0` with the original code, `DELETE 1` / `DELETE 1` / `DELETE 1` after switching cleanup to the owner connection, as now shown above.

- ☐ **Step 4: Run it**

Run: `pytest tests/test_rls_policy_regression.py -v` Expected: 3 passed. If the first test fails (Company A can see Company B with RLS on), Task 5's `tenant_select` policy is not in effect — check that `app_user` isn't accidentally the table owner (design decision #1). If the second test fails (the `UPDATE` succeeds instead of raising), `tenant_update`'s `WITH CHECK` is missing or incorrect — check design decision #6. If the third test fails, one of the 9

`current_setting(...)::uuid` casts in the migration is missing its `NULLIF` guard — check design decision #7.

- ☐ **Step 5: Commit**

```
git add backend/tests/test_rls_policy_regression.py
git commit -m "test: add RLS policy regression tests (disable/re-enable, cross-tenant reparent, GUC poisoning)"
```

Task 17: CI Pipeline

This is the literal Phase 0 exit criteria from `docs/09-roadmap-implementation-plan.md`: *"automated RLS isolation tests pass in CI."*

Files: - Create: `.github/workflows/backend-ci.yml`

- ☐ **Step 1: Write the workflow**

`.github/workflows/backend-ci.yml`:

```yaml
name: Backend CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_USER: postgres
          POSTGRES_PASSWORD: devpassword
          POSTGRES_DB: postgres
        ports:
          - 5432:5432
        options: >-
          --health-cmd "pg_isready -U postgres"
          --health-interval 5s
          --health-timeout 5s
          --health-retries 10

    steps:
      - uses: actions/checkout@v4
```

```
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install dependencies
        working-directory: backend
        run: pip install -e ".[dev]"

      - name: Run test suite (includes tenant-isolation and RLS
regression gates)
        working-directory: backend
        env:
          JWT_SECRET: ci-test-secret
          JWT_EXPIRE_MINUTES: "60"
          REDIS_URL: redis://localhost:6379/0
        run: pytest -v
```

- ☐ **Step 2: Verify locally with `act` or by pushing to a branch**

Run:

```
git checkout -b ci/phase-0-foundation
git push -u origin ci/phase-0-foundation
```

Expected: the "Backend CI" workflow run on GitHub Actions completes green, with every test file from Tasks 2–16 visible in the output — in particular `test_tenant_isolation.py` and `test_rls_policy_regression.py` passing is the actual exit criterion, not just a green checkmark.

- ☐ **Step 3: Commit**

```
git add .github/workflows/backend-ci.yml
git commit -m "ci: run backend test suite (incl. tenant-isolation
gate) on push and PR"
```

Task 18: Minimal Next.js Frontend Scaffold

Proves the Docker Compose topology from `docs/03-technical-architecture.md` Section 8 is wired correctly end-to-end. Full UI work starts in Phase 1 — this task deliberately does nothing more than confirm the frontend container can reach the backend.

Files: - Create: `frontend/package.json` - Create: `frontend/tsconfig.json` - Create: `frontend/next.config.ts` - Create: `frontend/app/layout.tsx` - Create: `frontend/app/page.tsx`

- ☐ **Step 1: Write `package.json`**

```
{
  "name": "builders-stream-frontend",
  "version": "0.1.0",
```

```
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start"
  },
  "dependencies": {
    "next": "16.0.0",
    "react": "19.0.0",
    "react-dom": "19.0.0"
  },
  "devDependencies": {
    "typescript": "5.6.3",
    "@types/node": "22.10.0",
    "@types/react": "19.0.0",
    "@types/react-dom": "19.0.0"
  }
}
```

- ☐ **Step 2: Write `tsconfig.json`**

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": false,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [{ "name": "next" }]
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx"],
  "exclude": ["node_modules"]
}
```

- ☐ **Step 3: Write `next.config.ts`**

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {};

export default nextConfig;
```

- ☐ **Step 4: Write `app/layout.tsx`**

```
export default function RootLayout({ children }: { children:
React.ReactNode }) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  );
}
```

- ☐ **Step 5: Write `app/page.tsx`**

```
async function getBackendHealth(): Promise<string> {
  try {
    const res = await
fetch(`${process.env.NEXT_PUBLIC_API_URL}/health`, { cache: "no-store"
});
    if (!res.ok) return `backend responded with ${res.status}`;
    const body = await res.json();
    return body.status;
  } catch {
    return "unreachable";
  }
}

export default async function Home() {
  const backendStatus = await getBackendHealth();
  return (
    <main>
      <h1>Builders Stream</h1>
      <p>Backend status: {backendStatus}</p>
    </main>
  );
}
```

- ☐ **Step 6: Verify the full stack together**

Run:

```
docker compose up -d
sleep 5
curl http://localhost:3000
```

Expected: HTML output containing `Backend status: ok`.

- ☐ **Step 7: Commit**

```
git add frontend/package.json frontend/tsconfig.json
frontend/next.config.ts frontend/app/
git commit -m "feat: add minimal Next.js frontend proving end-to-end
Docker Compose connectivity"
```

Task 19: Full-Stack End-to-End Regression Pass

Every prior task's tests run in-process against the app via `httpx.ASGITransport` — fast, but it never actually binds a port, never goes over real TCP, and never proves the two different hostname contexts (`postgres` inside the Docker network vs. `localhost` from the host — see the hostname note in Task 1) actually resolve correctly together. This task does that, once, as the final gate before Phase 0 is considered done. See "Regression Testing Policy" near the top of this document for why this exists as a separate layer from the per-task test runs.

Files: - Create: `scripts/e2e_smoke_test.py`

- ☐ **Step 1: Write the end-to-end smoke test script**

`scripts/e2e_smoke_test.py`:

```
"""Full-stack E2E regression check. Run against a live `docker compose
up -d`
stack — hits real HTTP ports, not the in-process ASGI transport the
pytest
suite uses. See docs/superpowers/plans/2026-07-07-phase-0-
foundation.md,
Task 19."""

import re
import sys
import time
import uuid

import httpx

BACKEND_URL = "http://localhost:8000"
FRONTEND_URL = "http://localhost:3000"
PASSWORD = "supersecret123"


def wait_for_backend(client: httpx.Client, timeout_seconds: int = 30)
-> None:
    deadline = time.time() + timeout_seconds
    last_error = None
    while time.time() < deadline:
        try:
            response = client.get("/health")
            if response.status_code == 200 and response.json() ==
{"status": "ok"}:
                return
        except httpx.ConnectError as exc:
            last_error = exc
        time.sleep(1)
```

```
    raise RuntimeError(f"Backend never became healthy within
{timeout_seconds}s: {last_error}")


def register(client: httpx.Client, company_name: str, email: str) ->
dict:
    response = client.post(
        "/auth/register",
        json={
            "company_name": company_name,
            "admin_full_name": "E2E Admin",
            "admin_email": email,
            "admin_password": PASSWORD,
        },
    )
    assert response.status_code == 201, f"register failed:
{response.status_code} {response.text}"
    return response.json()


def login(client: httpx.Client, email: str) -> dict:
    response = client.post("/auth/login", json={"email": email,
"password": PASSWORD})
    assert response.status_code == 200, f"login failed:
{response.status_code} {response.text}"
    return response.json()


def run() -> None:
    run_id = uuid.uuid4().hex[:8]  # unique suffix so repeated runs
don't collide on email uniqueness
    checks_passed = []

    with httpx.Client(base_url=BACKEND_URL, timeout=10.0) as client:
        wait_for_backend(client)
        checks_passed.append("backend /health reachable over real
HTTP")

        company_a = register(client, "E2E Company A", f"admin-a-
{run_id}@e2e.test")
        token_a = login(client, f"admin-a-{run_id}@e2e.test")
["access_token"]
        headers_a = {"Authorization": f"Bearer {token_a}"}
        checks_passed.append("company A registered and logged in over
real HTTP")

        own_company =
client.get(f"/companies/{company_a['company_id']}", headers=headers_a)
        assert own_company.status_code == 200, own_company.text
```

```
        checks_passed.append("company A can read its own company
record")

        company_b = register(client, "E2E Company B", f"admin-b-
{run_id}@e2e.test")

        cross_tenant =
client.get(f"/companies/{company_b['company_id']}", headers=headers_a)
        assert cross_tenant.status_code == 404, (
            f"CRITICAL: cross-tenant isolation failed over real
network — expected 404, "
            f"got {cross_tenant.status_code}: {cross_tenant.text}"
        )
        checks_passed.append("cross-tenant isolation holds over real
HTTP (company A cannot read company B)")

        child = client.post(
            f"/companies/{company_a['company_id']}/children",
            json={"name": "E2E Branch"},
            headers=headers_a,
        )
        assert child.status_code == 201, child.text
        child_id = child.json()["id"]
        checks_passed.append("nested child-company creation works over
real HTTP")

        child_read = client.get(f"/companies/{child_id}",
headers=headers_a)
        assert child_read.status_code == 200 and child_read.json()
["parent_id"] == company_a["company_id"]
        checks_passed.append("parent can read its own newly-created
child branch")

        invite_email = f"invitee-{run_id}@e2e.test"
        invite = client.post(
            "/invitations", json={"email": invite_email, "role":
"field_crew"}, headers=headers_a
        )
        assert invite.status_code == 201, invite.text
        checks_passed.append("invitation created over real HTTP")

        accept = client.post(
            f"/invitations/{invite.json()['id']}/accept",
            json={"full_name": "E2E Invitee", "password": PASSWORD},
        )
        assert accept.status_code == 200, accept.text
        checks_passed.append("invitation accepted over real HTTP")

        invitee_login = login(client, invite_email)
```

```
        assert invitee_login["default_company_id"] ==
company_a["company_id"]
        checks_passed.append("newly-invited user can log in and lands
in the correct company")

    with httpx.Client(timeout=10.0) as client:
        frontend_response = client.get(FRONTEND_URL)
        assert frontend_response.status_code == 200,
frontend_response.text
        # Next.js 16 / React 19 SSR inserts an HTML comment marker
between static
        # and dynamic text segments in a Server Component (design
decision #10),
        # so the raw body is literally "Backend status: <!-- -->ok",
not a
        # contiguous string — strip comments before asserting so this
doesn't
        # false-fail on correct output.
        rendered_text = re.sub(r"<!--.*?-->", "",
frontend_response.text)
        assert "Backend status: ok" in rendered_text, (
            f"Frontend did not report backend as healthy. Body:
{frontend_response.text[:500]}"
        )
        checks_passed.append("frontend container reaches backend
container over the Docker network and renders it")

    print(f"\n{'=' * 60}\nE2E SMOKE TEST:
{len(checks_passed)}/{len(checks_passed)} checks passed\n{'=' * 60}")
    for check in checks_passed:
        print(f"  PASS: {check}")


if __name__ == "__main__":
    try:
        run()
    except AssertionError as exc:
        print(f"\nFAIL: {exc}", file=sys.stderr)
        sys.exit(1)
    except Exception as exc:  # noqa: BLE001 - top-level smoke test,
want any failure to exit non-zero with context
        print(f"\nERROR: {exc}", file=sys.stderr)
        sys.exit(1)
```

- **☐ Step 2: Run the full pytest suite one final time**

Run:

```
cd backend
pytest -v
```

Expected: every test from every prior task passes — this is the last checkpoint before moving to real containers, per the Regression Testing Policy above. If anything fails here, stop and fix it before proceeding to Step 3; don't let an in-process regression get masked by the E2E pass below.

- ☐ **Step 3: Bring up the complete stack and apply migrations to it**

Run:

```
cd ..   # repo root
docker compose up -d --build
```

Expected: all four services (`postgres`, `redis`, `backend`, `frontend`) start. `postgres`/`redis` reach `healthy` (they have healthchecks); `backend`/`frontend` don't have healthchecks yet (a known, previously-flagged gap — Step 1 of the smoke script polls `/health` itself to compensate, so this doesn't block the check, but don't "fix" it as a side quest here — it's out of scope for this task).

Then apply migrations to the stack's real dev database (not the pytest suite's throwaway test database, which manages its own migrations independently):

```
cd backend
alembic upgrade head
```

This uses `MIGRATIONS_DATABASE_URL` from `.env`, which — per the Task 1 hostname note — correctly points at `localhost:5432` since Alembic runs on the host, not in a container.

- ☐ **Step 4: Run the E2E smoke test against the live stack**

Run:

```
cd ..   # repo root
python scripts/e2e_smoke_test.py
```

Expected: output ending in `E2E SMOKE TEST: 9/9 checks passed`, with every line prefixed `PASS:`. If the cross-tenant isolation check fails here (over real HTTP, real containers, real network) when the equivalent in-process test in Task 12 passes, that is a serious, `docker-compose.yml`/networking/environment-specific bug that the fast in-process tests structurally cannot catch — treat it as build-blocking, not a flaky test to retry.

- ☐ **Step 5: Tear down**

Run:

```
docker compose down
```

Note: this does not delete the `pgdata` volume (no `-v` flag) — intentionally, so local dev data survives between sessions. Don't add `-v` here without explicit instruction; removing a data volume is a destructive action outside this task's scope.

- ☐ **Step 6: Commit**

```
git add scripts/e2e_smoke_test.py
git commit -m "test: add full-stack end-to-end regression script (Task 19 gate)"
```

Phase 0 Exit Criteria Checklist

Cross-check against `docs/09-roadmap-implementation-plan.md`:

- ☒ Dockerized dev environment (PostgreSQL, Redis, backend, frontend) — Tasks 1, 18
- ☒ `companies`/`users`/`company_users` schema + nested hierarchy + `get_all_descendant_ids()` — Task 5
- ☒ RLS policies enabled and proven on Users & Company tables — Tasks 5, 12, 16
- ☒ `TenantMiddleware`: header/JWT extraction → contextvars → `SET LOCAL app.current_tenant` — Tasks 6, 11
- ☒ Auth: registration, login, invitations — Tasks 9, 10, 14
- ☒ Audit log table and a working write path — Tasks 5, 9, 13, 14, 15
- ☒ Exit criteria: automated RLS isolation tests pass in CI — Tasks 12, 16, 17
- ☒ Full-stack end-to-end regression pass against real containers over real HTTP (not just in-process ASGI tests) — Task 19

Explicitly Deferred (tracked, not forgotten)

- Refresh-token rotation and revocation (design decision #4)
- Multi-factor authentication for the Admin role (`docs/07-security-compliance.md` Section 1 flags this as needed before real subscriber data)
- Recursive-lookup caching in the JWT/session (only needed once a real company-tree depth/fan-out baseline exists — `docs/06-nonfunctional-requirements.md` Section 2)